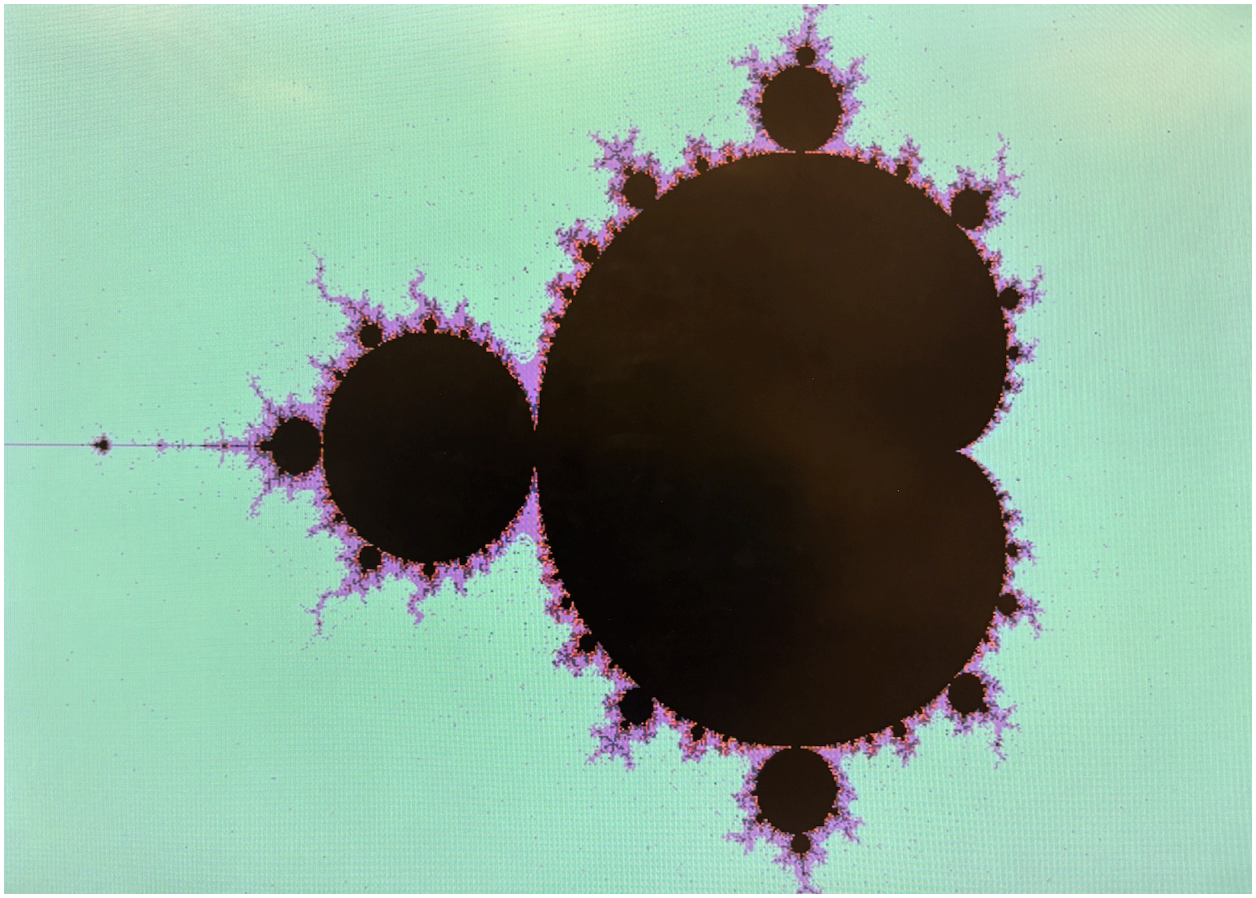
ECE 5760 Lab 2 Report

Mandelbrot Set Visualizer



Arnav M. | Nikhil S. | Zarif K.

03.04.2026

Full Set = 14.668ms

Number of Iterators = 24 Iterators

Misc Ops.: Main Cardioid + Left Circle

Introduction

In this lab, we implemented a Mandelbrot set visualizer. This entails logic calculating whether an input coordinate is in the set or not, connecting computation hardware and the VGA display together for visualization, and accelerating the computation to allow for a smooth and usable interface. The metric for acceleration was the rendering time for the full set, and our primary methods of acceleration were instantiating and distributing more Mandelbrot iterators and adding checks for known regions of the set. While the first lab provided us with experience using the different components of the board and programming the FPGA, this lab gave us our first glimpse into utilizing the FPGA and its associated components for hardware acceleration.

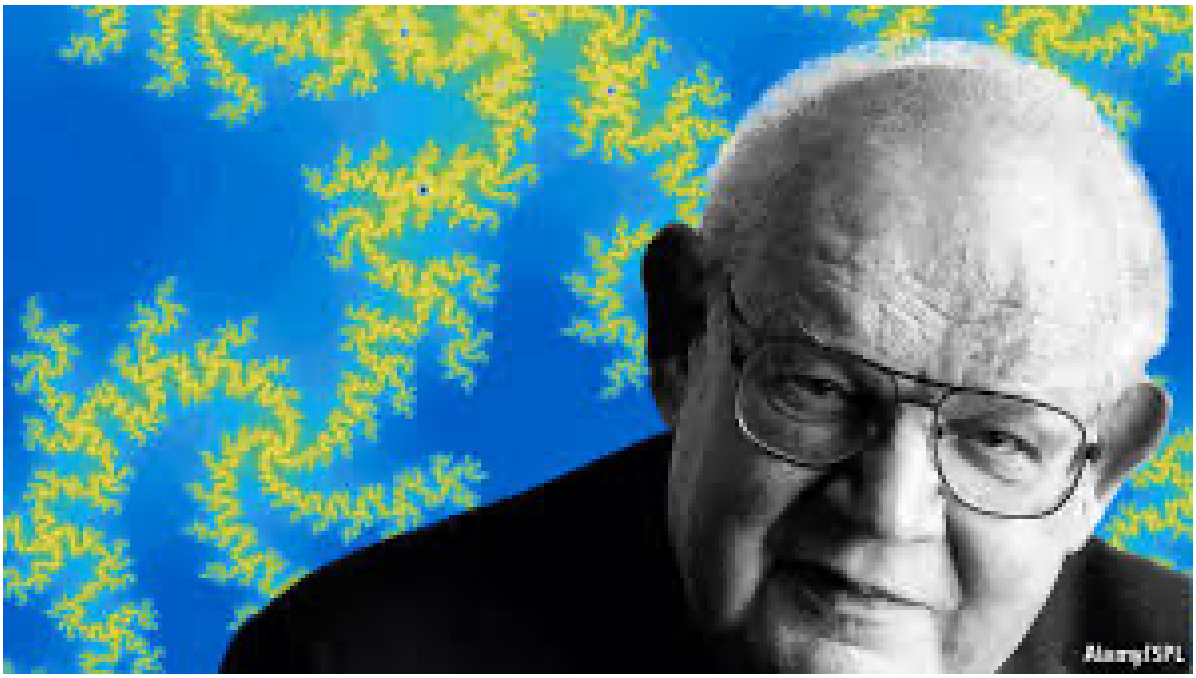


Figure 1: Mr. Mandelbrot + Zoomed in section of his mandelbrot set
https://www.economist.com/obituary/2010/10/21/benoit-mandelbrot

Mathematical Background

The Mandelbrot set consists of the complex points c such that the sequence shown below remains finite, with the initial condition $z_0 = 0$.

$$Z_{N+1} = Z_N^2 + c$$
$$Z_0 = 0$$

Figure 2: Mandelbrot set equations
https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot_Implementation.html

Plotting the points that are in the set results in the fascinating shape shown on the cover page. This plot has many interesting properties, notably that its edge is connected and that it is "infinitely rough." Zooming in on an edge will always reveal complex and unique structures, no matter how large the magnification.

Complex coordinates outside the Mandelbrot set result in a sequence that diverges to infinity. These points are determined to be outside the set if for some n less than a threshold number of iterations, the magnitude of z_n is greater than 2. Our implementation sets a threshold number of iterations of 1000, though this is a configurable parameter. A complex number is determined to be in the set if, within the threshold number of iterations, the magnitude of z_n never exceeds 2.

The sequence cannot easily be implemented in hardware in its base form. To simplify the implementation, the real parts and imaginary parts are divided into separate equations. After calculating the real and imaginary parts, the sum of the square of each component is computed to obtain the magnitude squared. The magnitude squared is then checked to see if it has exceeded 4, which is equivalent to checking if the magnitude has exceeded 2. Described in more detail in the RTL section, these squared values can be reused for the next iteration, halving the arithmetic and saving 2 multipliers per iteration. The equations calculated by the hardware are shown below.

$$Z_R^{N+1} = (Z_R^N)^2 - (Z_i^N)^2 + C_R$$
$$Z_i^{N+1} = 2Z_R^N Z_i^N + C_i$$

$$(Z_R^{N+1})^2 + (Z_i^{N+1})^2 > 4(?)$$

Figure 3: Equations Implemented in Hardware
https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot_Implementation.html

System Design

The primary purpose of the FPGA in this lab was to accelerate the calculations and rendering of the Mandelbrot set. The VGA display has dimensions 640x480 pixels, which initially maps to the complex plane from -2 to 1 on the real axis and -1 to 1 on the imaginary axis. While the pixel plane is always 640x480, the complex plane changes with zooming and panning, with each pixel mapped to exactly one complex coordinate. The FPGA contains instantiations of iterators, which perform the calculations, as well as the communication to memory which results in the coloring of pixels on the display. The HPS implements the user interface, which supports zooming and panning with minimal lag, allowing for the user to explore the set with precision and ease. The user interaction is done through the mouse, where mouse movement corresponds to panning and left and right clicks correspond to zooms in and out respectively. Zooming and panning is handled by changing the complex coordinates corresponding to the top left corner of the display and the distance in the complex plane between two pixels. These parameters are initialized and modified in the HPS code and communicated to the FPGA via PIO ports. In addition to implementing the user interface, the HPS can time the rendering—initiated by pressing Key[1]—which serves as our main metric for acceleration and print the coordinates corresponding to the corners of the display. Upon reset—initiated by pressing Key[0]—the FPGA sends a signal to the HPS to start timing, and after the rendering of the full Mandelbrot set is complete, the FPGA sends another signal to the HPS to end the timing.

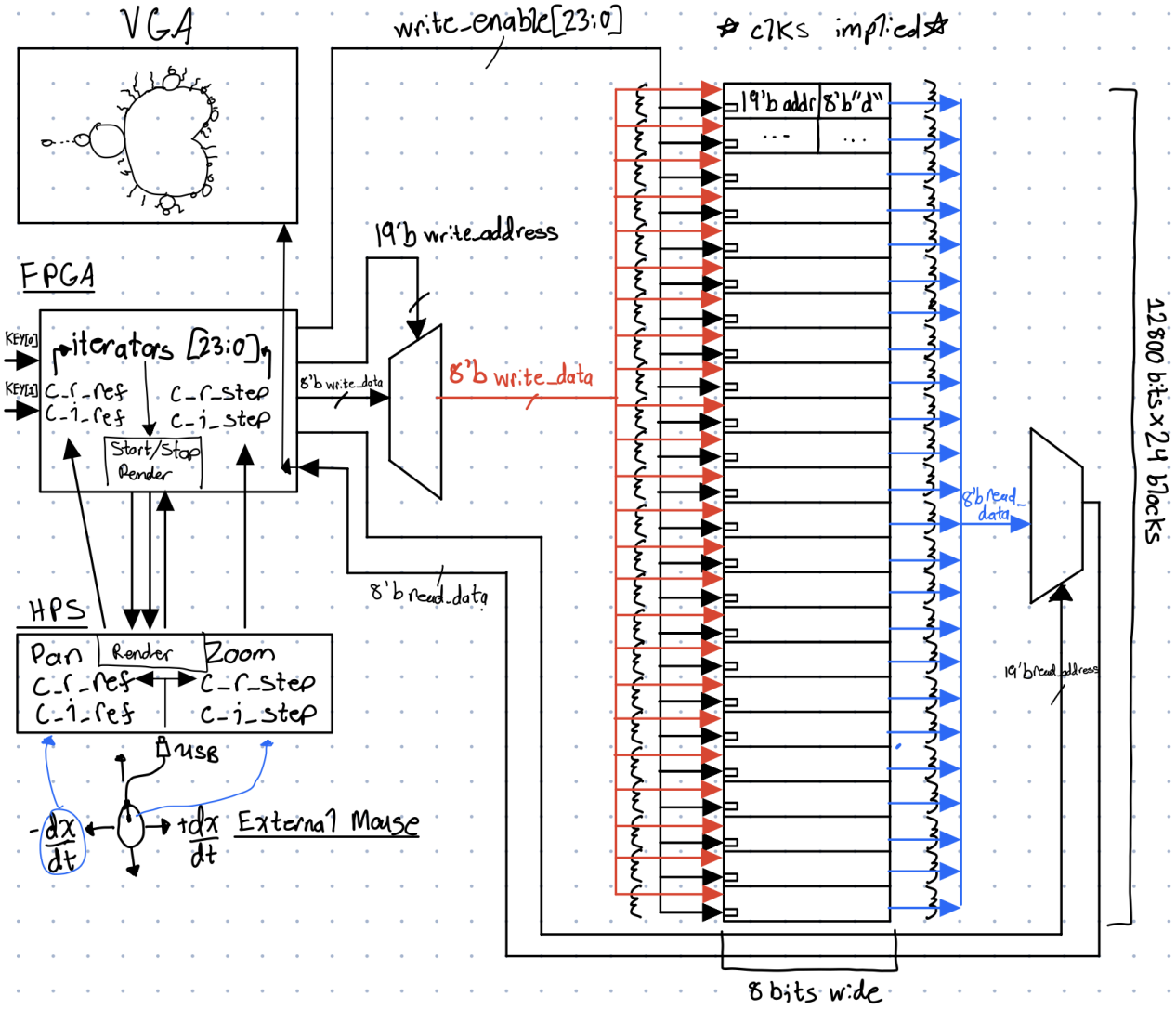


Figure 4: Architectural Flow from Hardware Up to Software Up to the Display

Fixed Point Arithmetic

We represented our values using a 4.23 fixed point representation. That is, each value is a 27 bit number with 4 integer bits and 23 fractional bits. In general, fixed point representation uses less resources and less cycles, while still allowing for fractional-level precision. To simplify our design and save resources on our FPGA side, we decided to use a somewhat high resolution fixed point representation to balance accuracy with efficiency. While using fixed point representation is more of a burden on the programmer, it is far more efficient in hardware. The 4.23 representation allows for high fractional precision in order to support high resolution at large magnification levels, but limits the integer range to +/- 8. This demonstrates the tradeoff between resolution and range exhibited by the usage of fixed point values. In this context, resolution is more important.

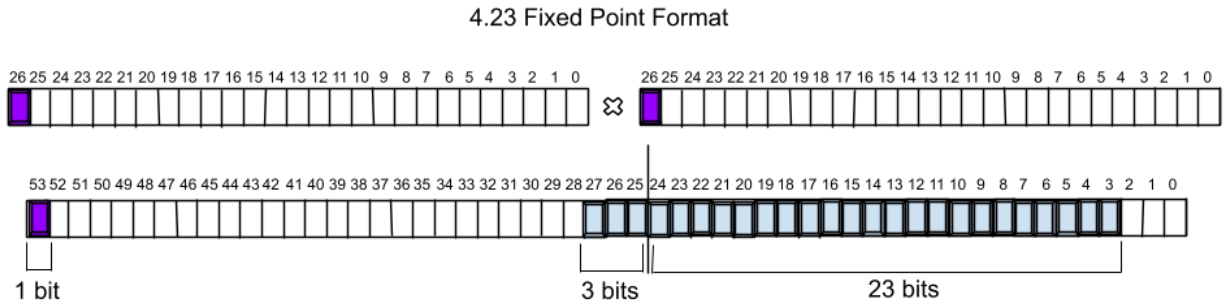


Figure 5: 4.23 Signed Fixed Point Representation

The solver instantiates the provided signed_mult module, which performs fixed-point multiplication between 27 bit values. The fact that the operands are 27 bits is crucial, as it will result in the compiler synthesizing a DSP block, which can handle one 27x27 multiplication. The initial multiplication results in an intermediate, 54-bit value. To obtain the final product, the sign bit (most significant bit) of the intermediate value is concatenated with the 26 bits surrounding the fractional point.

RTL Design

Iterator

The block diagram for the single iterator is shown below. The implementation makes use of the fixed-point multiplication module from the previous lab. Each iterator requires three instantiations of this multiplier, two to compute the squares and one to compute the product of z_r and z_i needed to calculate the next value of z_i. We once again use 27-bit values to enable the use of the DSPs for multiplication. We specifically use a 4.23 fixed point, which allows for the high precision necessary to enable significant magnification when zooming without little to no loss of resolution. After approximately **15 levels** of zooming in, the iterator is unable to discern between complex coordinates due to underflow, resulting in grainy blocks of color on the display. The extent that we are able to zoom is determined by the precision of our numeric representation.

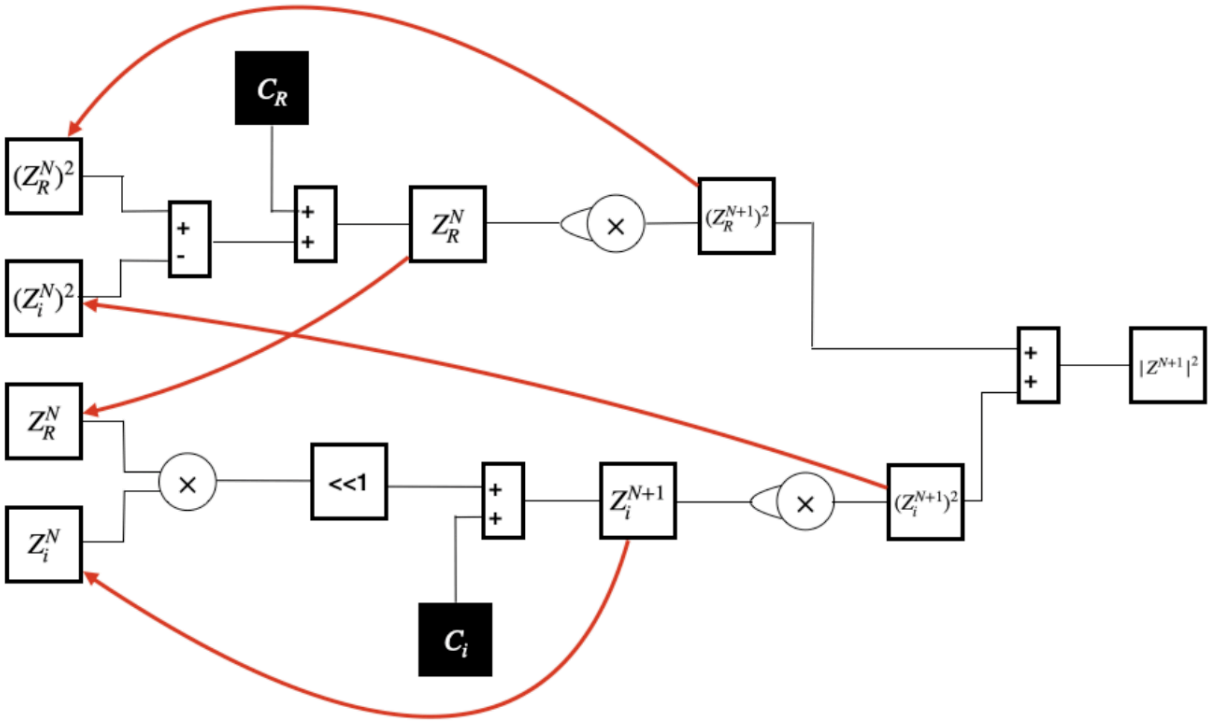


Figure 6: Implemented Iterator Block Diagram
(https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot_Implementation.html)

The iterator module takes in the complex number C as input, as well as clock, reset, and the maximum number of iterations. The module outputs a flag indicating that the calculation is completed, as well as the count for the number of iterations to reach completion. A variable keeps track of the squared magnitude of the value in the sequence. This variable is continuously checked to see if its magnitude is greater than 4, and the done flag is set if so. To avoid overflow issues, the individual values of z_I and z_R are checked for magnitudes greater than 2. This is discussed further in the Debugging section. 27-bit wires are used to keep track of the current and next values of z_I and z_R, as well as their squares. Upon reset, these four values and the count variable are set to 0. On every rising clock edge, the variables used to hold the current values get the next value (e.g. z_r <= z_r_next). With the exception of a few special cases for optimizations, the count variable is also incremented on every rising clock edge. In these cases, we conclude early that input is in the set, and "shortcut" the count to the maximum number of iterations. These special cases are discussed in more detail in the hardware optimization section.

Pixel Memory and VGA

The communication between the iterators and the VGA display was carried out through the use of M10k memory blocks. We implement a module called M10k_1000_8, which gets instantiated once per iterator. Along with clock and reset, this module takes in a data input, read address, and write address and enable, and has a data output as well. A register of some number of 8-bit values is created to represent the actual memory. On each rising clock edge, if the write is enabled the input data is written to the write address, otherwise the data at the read address is output. A synthesis directive(/* synthesis ramstyle = "no_rw_check, M10K" */) is used to tell the compiler to utilize the M10k blocks in hardware for this module.

In the state machine, each iterator has its own M10k memory block, where it writes the calculated colors for its assigned pixels. The system mapped the number of iterations to completion to a color, and then wrote this color to its memory block. The colors were represented by 8 bit RGB (3 bits R, 3 bits G, 2 bits B) values corresponding to the Commodore 64 Palette. This way, each iterator could independently traverse through their own coordinates and write the associated colors to memory, without needing to wait for each other or for the VGA system to process. The VGA driver would multiplex these M10k blocks, choosing a particular block based on the coordinate. Since each iterator had its own M10k block, the size of the M10k blocks was set by dividing the total number of pixels on the VGA display (640*480=307200) by the number of iterators. For example, in our final design of 24 iterators, each M10k block holds 12800 8-bit values.

The communication of the pixel information to the display is handled by the provided VGA driver. Since the VGA driver can only write one pixel at a time, only one instantiation is present. The provided VGA driver implementation is not changed. Our only task was setting the color_in input of the VGA driver, which tells the driver what color to write to the pixel represented by the variables next_x and next_y. This color was written to one of the M10k memory blocks from the state machine, and thus we use the next_x and next_y variables to index the M10k array and choose the particular block that corresponds to the iterator which writes that pixel. The mapping from pixel coordinate to iterator/M10k is given by next_x % 8 + 8*(next_y % 3). The VGA driver operates on its own clock, vga_pll, which runs at 25 MHz. The state machine operates on the M10k_pll clock, which operates at a faster but in-phase frequency.

State Machine

The iterators and memory blocks were integrated together in a state machine. A high-level diagram of this state machine is shown in figure 7.

The main method for acceleration was to increase the number of instantiations of the iterator and have them calculate a different subset of the plane in parallel. Successful parallelization involved distributing the iterators evenly throughout the plane. Since each iterator involves three 27-bit multiplies, and thus three DSPs, we could theoretically have up to 29 iterators running in parallel, utilizing all 87 DSPs. We experimented with 1, 2, 16, and 24 parallel iterators. For the latter two cases, the iterators were arranged in a 4x4 and 8x3 rectangle replicated across the plane, allowing for even distribution of iterators and thus even load balancing. Non-optimal distribution of iterators would result in one iterator taking significantly longer than the others. The time to render is dictated by the time taken for all pixel colors to be calculated and written, so having one particularly slow iterator results in unsuccessful acceleration. The organization of iterators in our final design is shown in figure 9.

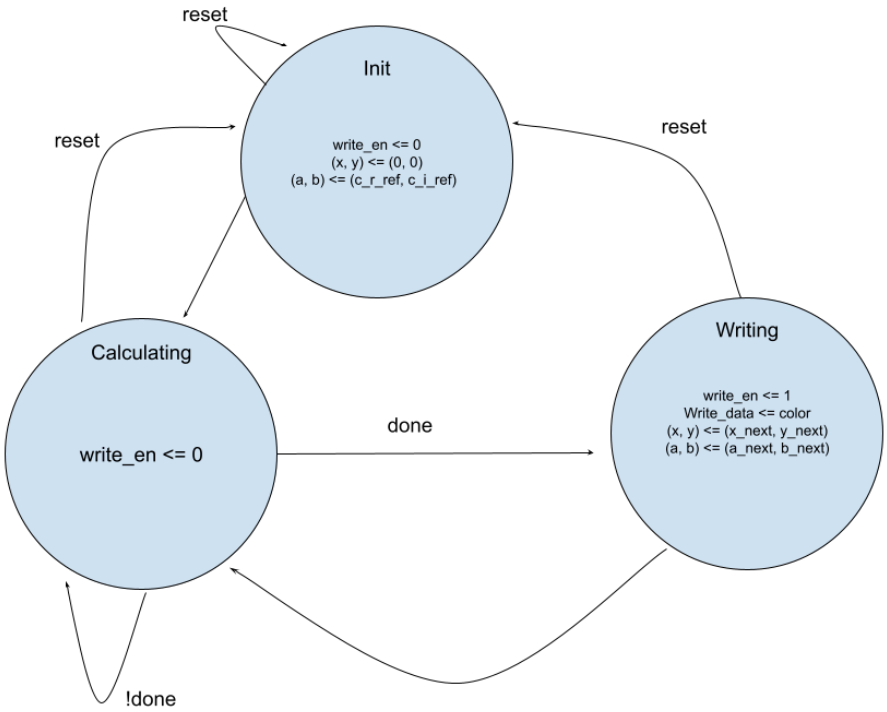


Figure 7. Full System FSM

To instantiate the 24 iterators and M10k blocks, we made use of *generate* blocks. *Generate* blocks allow us to encapsulate module instantiations inside for loops. This helps make the code more concise, and is possible because each iterator operates independently and in parallel.

Outside of the iterator module, the system needs to not only map VGA pixels to complex coordinates, but also map complex coordinates to iterators. Rather than conducting a conversion from the pixel plane to the complex plane, the system keeps track of the current top left corner in

the coordinate plane, which is mapped to (0, 0) in the pixel plane, and the step size in each direction in the complex coordinate plane corresponding to one pixel. The top left corner would change with panning, while both the top left corner and the step size would change with zooming. Since these actions were captured through the HPS, the corner and step size were communicated via PIO ports from the HPS to the FPGA. The mapping from the pixel to the complex plane is shown in figure 8.

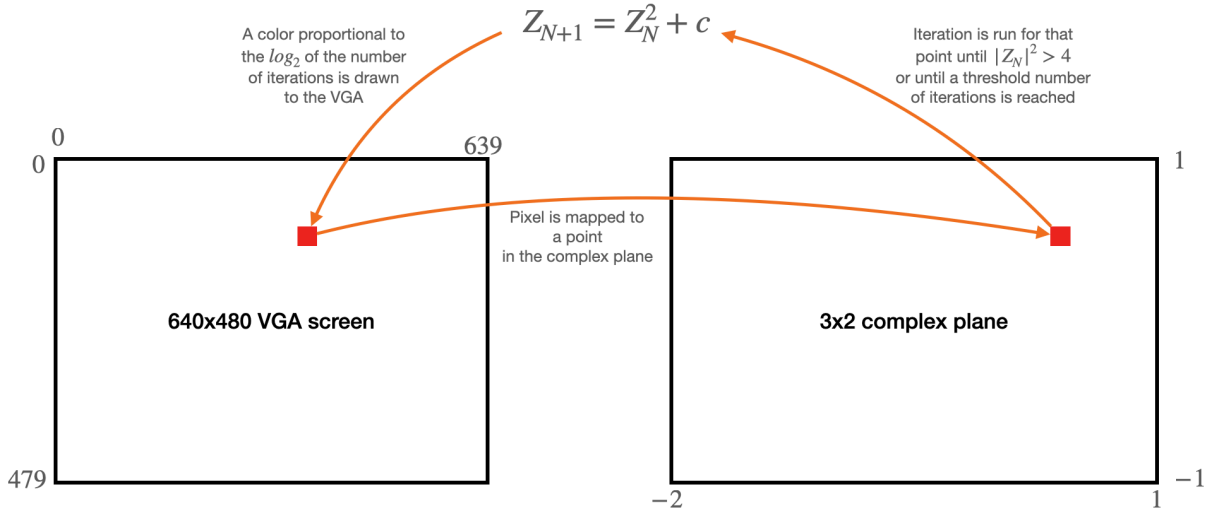


Figure 8: Pixel plane to complex plane mapping
https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot_Implementation.html

Upon reset, each iterator needs to reset to its starting pixel and complex coordinate. Each iterator has an ID 0-23. Modulo and division logic is necessary to identify the starting pixel and coordinate for each iterator. Specifically, an iterator with ID ii will have a starting x coordinate of ii mod 8 and a starting y coordinate of ii / 8. It is important to note that '/' implements integer division. Using modulo and division operators allows the structure to be generalizeable to other amounts of iterators, but if the rectangle of iterators has dimensions that are powers of 2, this logic would be more efficiently implemented using indexing and shifts.

The reset logic in the coordinate plane is similar but slightly more involved. To support zooming and panning, the coordinate corresponding to the top left corner of the VGA display is captured by variables c_r_ref and c_i_ref, which are calculated and communicated by the HPS via PIO ports. Additionally, the step size in each direction is held by variables c_r_step and c_i_step, also calculated by the HPS. The starting position is calculated as an offset added to c_r_ref and subtracted from c_i_ref. The offset is subtracted, rather than added, since the y axis on the pixel plane increases going down. Depending on the configuration of iterators, each iterator may jump more than one pixel to get to its next assigned coordinate. In the case of the 8x3 arrangement of iterators, each iterator jumps 8 pixels in the x direction to get to its next, and 3 pixels in the y

direction once it has reached the end of its row. The variables c_r_step and c_i_step correspond to these jumps. Before any zooming, c_r_step and c_i_step correspond to 8 and 3 pixels in the complex plane respectively. Since the initial 640x480 plane corresponds to -2 to 1 in the real axis and -1 to 1 in the imaginary axis, c_r_step and c_i_step are initially set to 24/640 and 6/480 respectively. When initializing the iterators to their starting complex coordinate, the offset from the pixel plane is multiplied by c_r_step divided by 8 and c_i_step divided by 3 in order to get the distance corresponding to one pixel.

In normal operation, the system checks the done flag from each iterator on each rising clock edge. If a particular iterator is done at that clock edge, the corresponding color must be written to that iterator's M10k block and the next coordinates for that iterator in both the pixel and complex plane must be calculated. When an iterator is done, the write enable to its M10k block is set and the address to write the color is calculated. The pixel address is mapped to the address in M10k memory by dividing the x coordinate by 8 and adding 80 times the y coordinate divided by 3. Each iterator has 80 pixels in each of its rows. Pixels corresponding to coordinates in the set are colored black, and other pixels are colored based on the base-2 log of the number of iterations taken for that point.

To calculate the next coordinate for each iterator, we must check if the iterator is at its last assigned pixel in the current row. Each iterator's last coordinate is found by applying the same offsets as the reset conditions to 632 in the x axis and 477 in the y axis. Each iterator moves across the row before moving to its next row. Thus, if the iterator is not at its last pixel in the row, its next coordinate is simply its current x coordinate plus 8. If it is at its last pixel in the row, then it resets to the x coordinate of its starting pixel. The y coordinate only changes if the iterator is at its last pixel in the row. If the iterator is not at its very last pixel, then the next y coordinate is found by adding 3 to the current y coordinate. If it is at its last coordinate, the y coordinate is also reset to its starting value.

When a particular iterator is done writing the colors for all of its pixels to memory, it updates its corresponding bit in a 24-bit status register. When all 24 bits in this register are set, the FPGA raises the signal used by the HPS to end timing. In the state machine, we check the x_coord and y_coord variables to see if the iterator is at its last pixel, and set the corresponding status bit if so.

Hardware Optimization Considerations

Realizing performance boosts from parallelization, optimizations began with instantiation of 1 extra iterator. We then implemented a pattern in which every other pixel is drawn by one iterator, starting from either of one origin points.

	0	1	2	3	4	5	6	7			632	633	634	635	636	637	638	639
0	0	1	2	3	4	5	6	7	.	.	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15	.	.	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23	.	.	16	17	18	19	20	21	22	23
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.		
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.		
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.		
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.		
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.		
477	0	1	2	3	4	5	6	7	.	.	0	1	2	3	4	5	6	7
478	8	9	10	11	12	13	14	15	.	.	8	9	10	11	12	13	14	15
479	16	17	18	19	20	21	22	23			16	17	18	19	20	21	22	23

Figure 9. Iterator plotting pattern

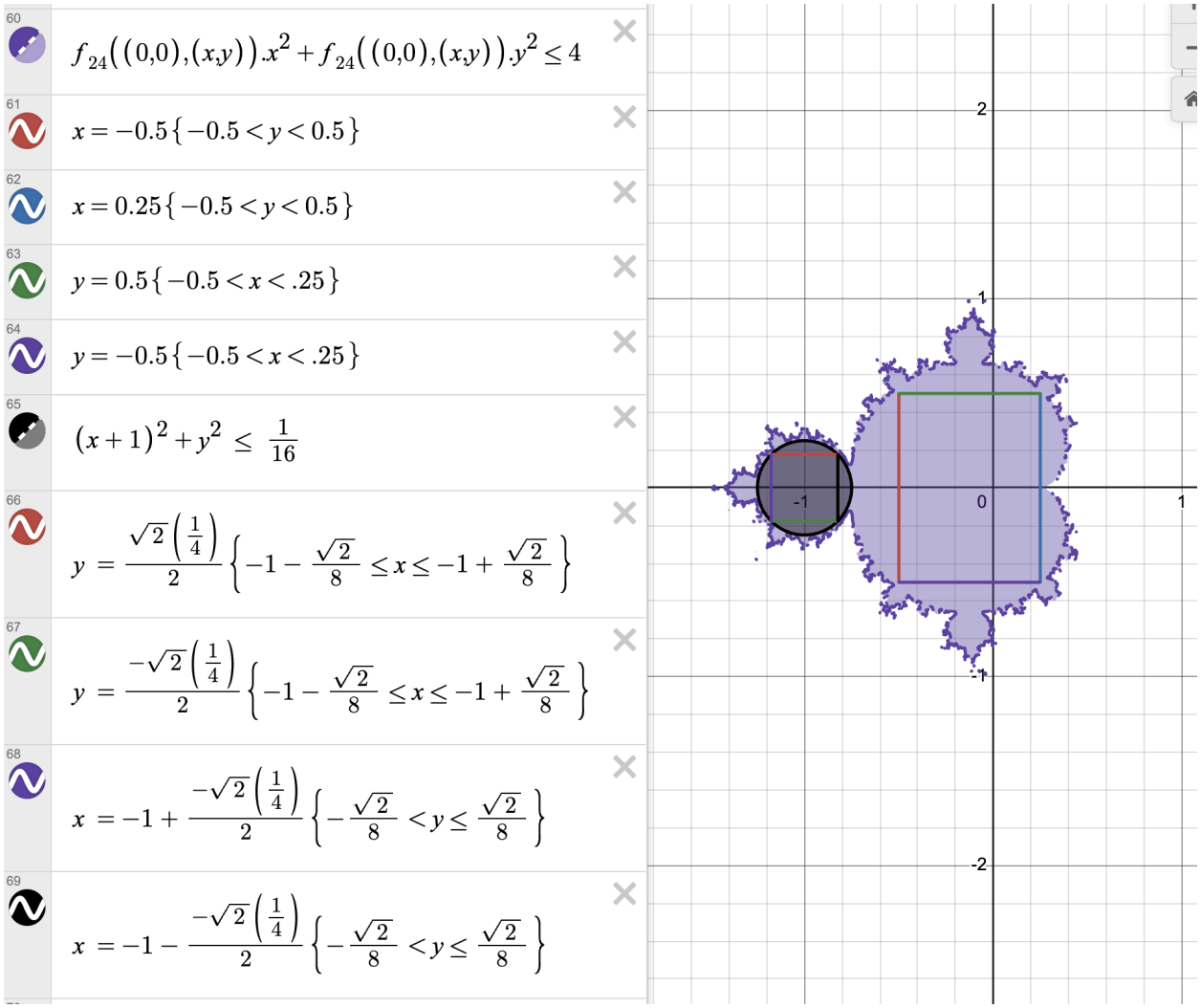


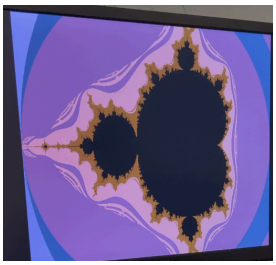
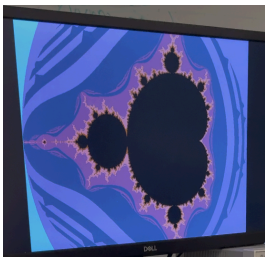
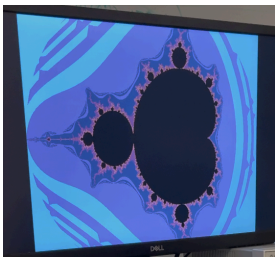
Figure 10. Desmos Graph of Mandelbrot Set; Rectangle optimizations visualized
https://www.desmos.com/calculator/l0qbhwn7pf

After implementing rendering with 16 iterators, we desired clever optimizations. There are two significant regions where all points are in the set: the main cardioid and the period-2 bulb. Since concluding that a point is in the set entails counting up to the maximum number of iterations, significant time could be saved by checking if a point is within one of these known regions and ending the iterations if so. The period-2 disk is a circle, so checking if points are within this region involves two more multipliers per iterator. On 16 iterators, this reduced the rendering time from 50 ms to 43 ms, but the additional two multipliers per iterator from implementing the circle equation led to 80 DSPs being used. While the equation for the main cardioid is known, implementing the check exactly would exceed the maximum number of DSPs. Thus, we decided that using a rectangle within the region as our check would result in significant optimization without any multipliers. Doing this reduced the rendering time from 43 ms to 19.5 ms. In order

to allow for more iterators, we replaced the check for the period-2 bulb with a rectangle within that region as well. These two optimizations, while very simple in code, enabled significant acceleration. The rectangles overlaid with an existing plot of the set in Desmos is shown in Figure 10.

Editing Max Number of Iterations

The switches on the FPGA board were used to control the maximum number of iterations used to determine whether a point is in the Mandelbrot set. All ten switches were used, with each switch corresponding to a power-of-two bit in a binary number, forming a 10-bit value allowing the iteration count to range from 0 to 1023. Below are sets produced using different values for this variable.

Max Iterations: 31	Max Iterations: 63	Max Iterations: 127	Max Iterations: 255
			
Fig 11	Fig 12	Fig 13	Fig 14

We also discovered through this testing that render time increase was correlated to decreasing the maximum number of iterations. It takes less time for an iterator to be done at a lower maximum of iterations at the cost of accuracy. At a maximum of 31 iterations, we are still able to decipher the general shape of the mandelbrot set. Any lower and the canonical shape is lost, eventually leading to a complete black screen.

Software Design

Architectural Approach

After completing the rendering of the full Mandelbrot set via one iterator, we then implemented interactivity with the plot. To streamline production of this feature, C code was used to send control signals over memory mapped PIO ports. Both zoom and pan are controlled and calculated on the HPS side of our system.

Below are the variables calculated based on user mouse input:

- c_r_step
- c_i_step
- c_r_ref
- c_i_ref

Zoom scales the step size across both complex axes by 2 on requested zoom input. Regardless of the number of iterators in HW or pattern of render, the following holds: on left click, c_r_step and c_i_step are divided by 2 to render finer points, thereby also displaying ¼ of its previous screen; on right click, c_r_step and c_i_step are multiplied by 2 to render courser points, thereby displaying 4 times more of the broader plot to be rendered.

One neat trick to address a concern of unintentional rapid zooming on a click was to debounce mouse clicks. This required tracking a previous state of the mouse and the current state of the mouse to ensure only one click is processed at a time, even if the user holds down the click.

Panning edits the reference point to start rendering from. Mouse movement shifts the entire Mandelbrot Set in both the x and y directions.. Here, we tracked whether there is negative or positive movement in both axes to simply increment the c_r_ref and c_i_ref at a constant rate.

Monitoring mouse input required reading relative mouse speed over its own readable file from our OS memory. We were able to read speed in the x and y direction, as well as left and right clicks.

Render Timing

A high signal is sent to the HPS over the Start_Rendering PIO port after a Key[0] or Key[1] click to start capturing time. The HPS then knows to start recording the start time by getting the time of day. We then wait until a different high signal is sent to the HPS over the Done_Rendering PIO port after all iterators have computed their results to end capturing time. We then get the

new time of the day to compute the difference of the end time and start time. Afterwards, this value is converted to milliseconds to be displayed to the terminal. Before we loop again, we also wait for the Start_Rendering PIO to return to 0 to avoid immediately triggering again on the same signal.

Multi Threaded approach

In general, GUIs require multithreading for smooth execution of multiple tasks. Putting hardware and software together, multithreading was used to divide responsibility of the mouse and render timing. We utilized the “pthread” library to split our render timing and mouse movement processing into two processing threads.

Compared to our first lab, the FPGA takes more responsibility for plotting. This proves for more acceleration based on external computation, analysis that will be discussed later in our results.

Note that the PIO labeled “MOUSE_ZOOM_PIO” is never used. An original implementation of sending a users zoom requests directly to the FPGA for processing was replaced with a C-side implementation to conserve resources for rendering acceleration. Additionally, “MOUSE_X” AND “MOUSE_Y” are not utilized, as troubles of clock domain crossing occur when aligning mouse monitoring on the C-side with panning on the FPGA side. A C-side only approach was used for panning in the end to resolve this issue.

Type Consistency

As our system is not able to discern between fixed and floating point, a macro on the C-side was created to enforce type consistency. The C program would pull the 32 bit integers from memory (the sign extended 4.23 fixed point numbers), and this value would be passed to a macro to convert to floating point for ease of use in C. This macro involved either multiplying by 2^23 to convert to float, or dividing by 2^23 to convert to fixed point. The order of operations involved in this goes as follows:

- Capture the value of 2^23
- Multiply the input by 2^23
- Cast to our created type “fix”

This is equivalent to “fix result = (fix)(a * 8388608);” where fix is a typedef for int.
With this recognizable form of values, we then were able to interpret and utilize these numbers for plotting and editing of parameters.

Platform Designer

The Platform Designer, formerly known as Qsys, tool in Quartus aided in making connections between the FPGA and the HPS on the AXI bus. Communication between them is implemented via memory-mapped IO (MMIO) where virtual address offsets in the HPS are assigned to the FPGA registers for the ARM core to be able to access the FPGA registers.

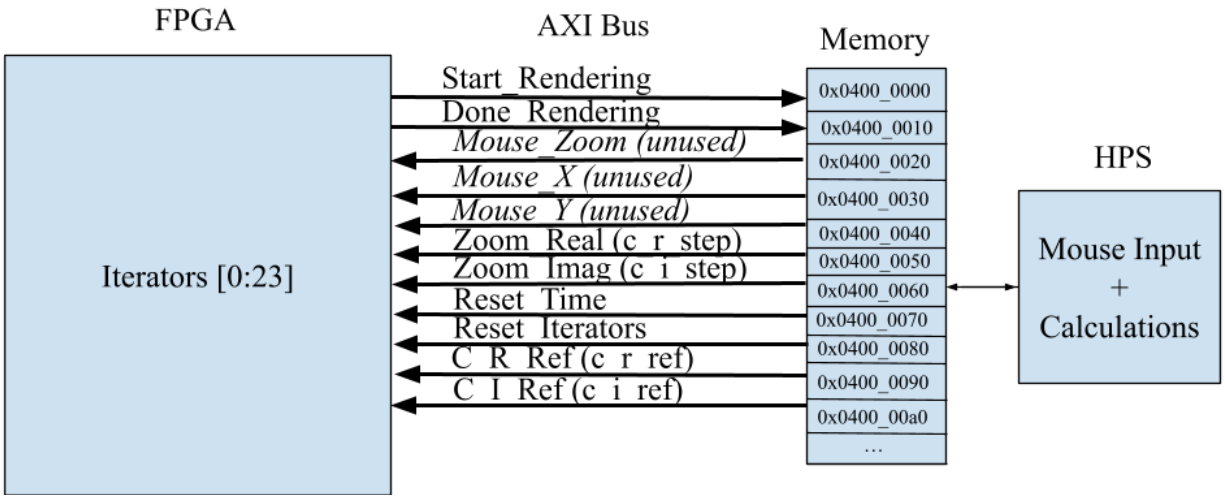


Figure 15: FPGA and HPS System Diagram

Several Parallel I/O (PIO) ports were created to send key rendering information to the HPS and to the FPGA. Again, in order to conserve resources crucial for accelerating render of the Mandelbrot set, only light calculations were outsourced to the HPS side. The HPS primarily handles user interaction, which includes some basic parameter calculation. start_rendering, done_rendering, and reset_time were used in aid of capturing rendering time. When rendering begins, the FPGA drives start_rendering, and once all iterators have completed, the FPGA drives done_rendering, allowing the HPS to measure the time between when these two signals are driven. reset_time allows the HPS to reset timing measurement before starting a new render, and reset_iterators is used when the FPGA is reset to reinitialize the complex plane parameters.

Additional PIOs were used in aid of pan and zoom. The HPS computes the updated complex plane parameters—reference coordinates c_r_ref and c_i_ref and step sizes c_r_step and c_i_step—and writes them to memory, which the FPGA can read through registers. Note that x_mouse, y_mouse, zoom_mouse are unused PIO ports in the current implementation.

Testing Methods

Python Simulation

Before working on the hardware implementation of the solver, we used a Python implementation provided to understand the behavior of the system. This Python model served as our golden reference model. After first implementing the iterator module, we provided inputs to both the Python and Verilog implementations and checked not only that the inputs were correctly classified as in the set or not, but that the two implementations took the same amount of iterations to come to the conclusion.

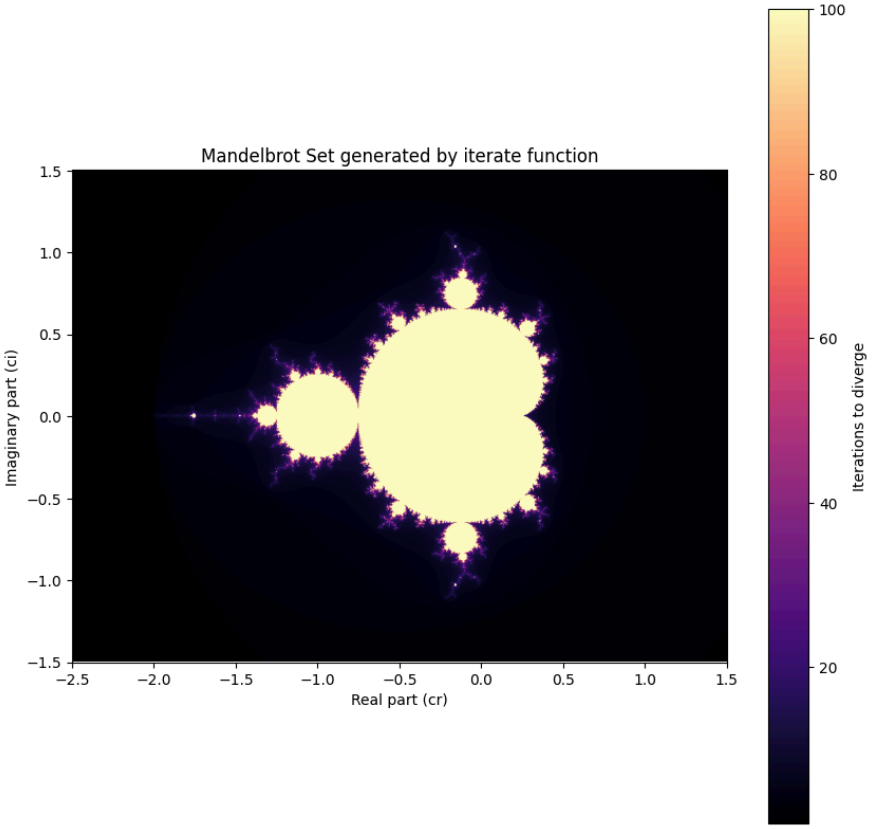


Figure 16: Python Simulation Rendering
https://colab.research.google.com/drive/1sEoqDTAy-Hmy0DL8UM6cxjAtCky3Cxzm#scrollTo=cb6b15db

Verilog Simulation

Once we had an initial implementation of the solver, we used ModelSim to verify the functional correctness. ModelSim simulates our synthesized hardware design, allowing us to avoid the long

compile times from programming the FPGA for debugging. We used ModelSim to ensure the classification of an input as well as the number of iterations – number of cycles the solver ran for – matched the number of iterations from our Python golden reference model.

Common Bugs and Debugging Methods

One initial bug identified was that some complex coordinates of large magnitude were classified as in the set. This appeared on the plot as seemingly random pixels being colored around the set. We found that this was a result of overflow, as the large coordinates would result in the mag_squared variable being interpreted as a small value. We resolved this issue by checking not just the squared magnitude, but also the individual z_i and z_r values to see if they were greater than 2, setting the done flag if so.

When first attempting the state machine, one challenge was resetting each iterator before it moved on to its next pixel. We tried to manually set the reset signal to the iterators from the state machine after it completes all of its pixels. However, this resulted in a timing mismatch, as the synchronous reset was processed after the asynchronous input was received. We resolved this by having each iterator reset when its done flag is raised. While this would cause each iterator to immediately start recalculating after completing one calculation, our state machine ensures that the iterator will move on to its next coordinate in time, avoiding risk of recalculating the same coordinate repeatedly.

When testing our first implementation of the 16-iterator system, we found that the plot appeared grainy, as if the pixels were colored in blocks. We were initially unsure if the bug was in the code writing to memory or the code reading from memory. To help isolate the problem, we modified the code such that each iterator was assigned a unique color, and would color all of its pixels accordingly. Thus, we would expect to see blocks of 16 pixels of different colors across the display. Instead, we continued to see blocks of colors. We ultimately resolved the bug by fixing a math error in some of the address calculations within the state machine.

Our initial attempts at the user interface involved a mix of handling from the HPS and the FPGA. The HPS captured the mouse movements and passed those through PIO ports to the FPGA, which interpreted these values and attempted to pan or zoom accordingly. We found that this resulted in unstable behavior, likely due to synchronization issues between the HPS and FPGA logic. To simplify the design and use less hardware resources, we decided to have the entire user interface implemented in the HPS, directly updating variables in the RTL design through PIO.

After successfully rendering the full set, we found that panning and zooming far out could result in other Mandelbrot sets appearing in a periodic manner. We again concluded that this was an

overflow issue. Since our representation only had 4 integer bits, numbers do not have to be very large to cause overflow behavior. To resolve this issue, we clamped our maximum zoom out on the software to avoid computing the set smaller than the full representation. Realistically the user desires only to see that of which is given, no further out.

Results

Performance Comparison

Implementation	Compute Time (ms)	Speedup
HPS (Baseline)	883	1.00x
FPGA (1 solver)	794	1.12x
FPGA (2 solvers, column-wise)	397	2.22x
FPGA (16 solvers: 4x4)	50	17.66x
FPGA (16 solvers: 4x4; period-2 bulb circle opt)	43	20.53x
FPGA (16 solvers: 4x4; period-2 bulb circle opt; cardioid rectangle opt)	19.5	45.28x
FPGA (24 solvers: 8x3; cardioid rectangle opt)	17.5	50.46x
FPGA (24 solvers: 8x3; cardioid rectangle opt; period-2 bulb rectangle opt)	14.66	58.87x

Table 1. Solution time for the full set versus number of solver pipelines and optimizations

Our system went through many version changes, with the basis that we will beat the render timing of a general CPU producing the mandelbrot set. Through each design change, we capture the trend in decreasing compute time. One notable trend was a linear decrease in render time as the number of iterators increased.

Solution Time depends on number of solvers:

- 1 iterator took 800 ms
- 2 iterators took 400 ms
- 16 iterators now takes 50 ms

Therefore there’s approximately this rule: render time = (800ms) / num_iterators

After reaching 16 iterators, we opted for clever optimizations that drastically reduced render time. In the end, we reached nearly 60x render time improvement compared to our C baseline code.

Even more interestingly, the C code also has the same optimizations of auto-completing iterators if they lie in the range of the cardioid and Period 2 circle, but had significantly longer render time compared to the same optimizations we added to our FPGA implementation. This shows the true power of using FPGAs hand in hand with system and software development!

Additionally, we zone in on specific regions of the Mandelbrot set for alternate key metrics:

Region	Compute Time (ms)
x=[−2,1], y=[−1,1] (Full Set)	14.66
x=[−0.758,−0.75], y=[0.05,0.06] (Seahorse Region)	57
x=[−1.45,−1.3], y=[−0.07,0.07] (Bulb to the left of Period 2 Bulb)	69

Table 2. Compute times for various regions of the Mandelbrot set close to the coordinates, 1000 max iterations

Our rectangle optimization in both the cardioid and the period 2 bulb do not extend and are touching the “seahorse” region, so we expect a higher render time. We also did not optimize the mini bulb to the left of the period 2 circle, so we also expected to encounter a longer rendering time! These render times could have been minimized by reading a hardware circle and cardioid equation, but since those equations require multiplication (the equivalent of expanded exponents to multiplying a term with itself) we ultimately decided to conserve our DSP blocks to create more iterators.

Favorite Regions

Below are regions we found interesting as they were mathematically generated.

Fig 17. “Floral Mandelbrot” at x=[-0.564133, -0.563950], y=[-0.640849, -0.640727]
Peculiar fractals that keep branching out of a moat that surround an even smaller mandelbrot set!

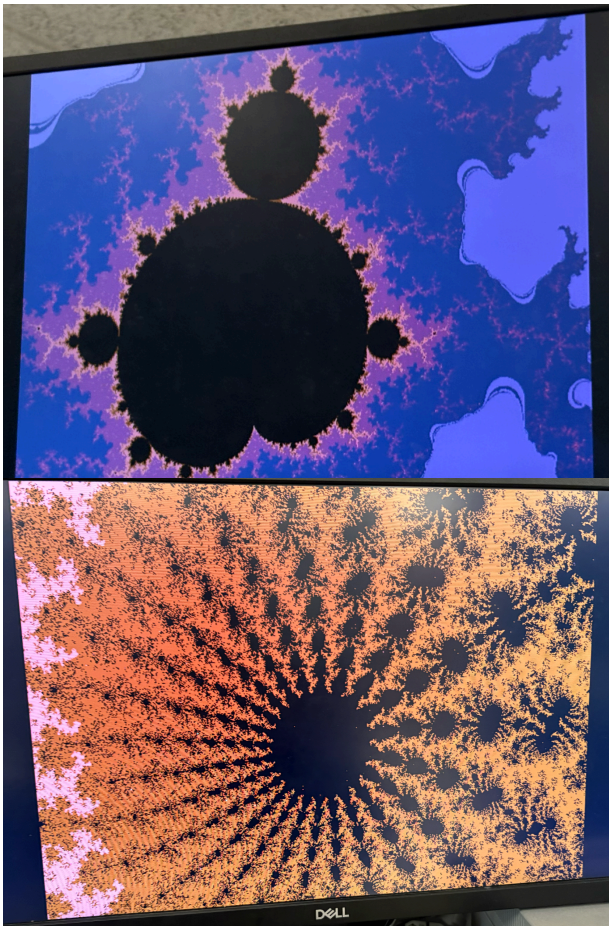


Fig 18. "Crooked Mandelbrot" at x=[0.429819, 0.435768], y=[0.225715, 0.229621]
What looks like a smaller mandelbrot set, but its cardioid is uneven!

Fig 19. "Ant colony" at x=[0.245434, 0.245617], y=[-0.557998, -0.557876]
On the righthand side of this image, we can pick out a creature with 3 segments and tiny legs branching out to be almost ant-like! They become blobbed up and are sent into a 2D Ant hole.

We hope you enjoy our favorite regions as much as we do! It is splendid how a natural force produces such beautiful art!

Resource Utilization

From the Quartus resource utilization report, it is clear that the resource requirements are largely from the instantiation of numerous rendering pipelines. At the top level, the DE1_SOC_Computer uses approximately 11,328 Adaptive Logic Modules (ALMs), 9851 logic registers, 72 DSP blocks, and 384 M10K memory blocks. An approximation of one rendering pipeline's ALM usage is close to 11,328 ALM / 24 iterator ~ 400 ALM / iterator. It is evident that chip area quickly expands as we increase parallelization. We are limited by the fixed area of the chip, but for the purposes of this lab we did not reach this limit.

Addition of extra iterators required extra memory blocks to store new pixel color values. Though we conceptually segment our memory into 24 sections for each of the iterators, our tools create

contiguous M10K blocks to partition memory as one whole memory block, therefore resulting in near 100% M10K block usage.

Resource	**Usage**
ALM	11,328 / 32,070 (35%)
PLLs	2/6
M10K Blocks	384 / 397 (97%)
DSP	72/87 (83%)
Registers	9851

Table 3: Design Resource Utilization for Final System of 24 Iterators and 2 Rectangles

Chip Planner

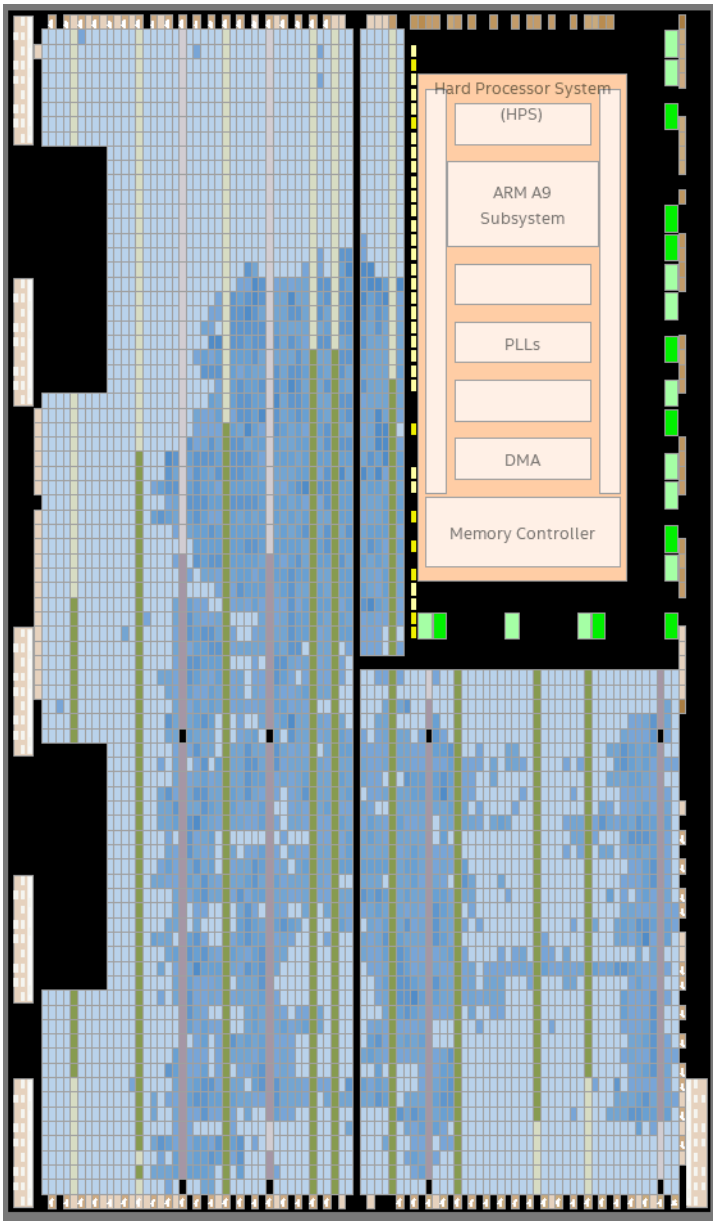


Figure 20: Chip Planner View of Design

Green blocks are RAM elements (M10K blocks). Light purple blocks are DSP blocks. Brown blocks along the top right edges of the HPS are PIO ports. Longer grouped sections along the left edges are PLLs. There is a notable amount of DSP block usage in our chip plan as we expect from the numerous multiplications, which are then sent out to the iterators. The 24 parallel iterators, then, take up a notable amount of area on the chip, but just enough to add more logic as necessary for future optimizations. This is the cost of parallelization inevitably.

Critical Path Analysis

Multicorner Timing Analysis Summary

<<Filter>>

	Clock	Setup
1	Worst-case Slack	-36.247
1	CLOCK_50	N/A
2	Computer_System:The_System|Computer_Sys...nst|hps_sdram_pll:pll|afi_clk_write_clk	1.727
3	Computer_System:The_System|Computer_Sys...dram_pll:pll|pll_write_clk_dq_write_clk	N/A
4	The_System|m10k_pll|altera_pll_i|general[0].gpll~PLL_OUTPUT_COUNTER|divclk	-36.247
5	The_System|system_pll|sys_pll|altera_pll_i|general[0].gpll~FRACTIONAL_PLL|vcoph[0]	N/A
6	The_System|system_pll|sys_pll|altera_pl...neral[0].gpll~PLL_OUTPUT_COUNTER|divclk	1.104
7	The_System|vga_pio|altera_pll_i|general[0].gpll~FRACTIONAL_PLL|vcoph[0]	N/A
8	The_System|vga_pio|altera_pll_i|general[0].gpll~PLL_OUTPUT_COUNTER|divclk	-0.430
9	clk_vga	-17.317
2	Design-wide TNS	-85540.031
1	CLOCK_50	N/A
2	Computer_System:The_System|Computer_Sys...nst|hps_sdram_pll:pll|afi_clk_write_clk	0.000
3	Computer_System:The_System|Computer_Sys...dram_pll:pll|pll_write_clk_dq_write_clk	N/A
4	The_System|m10k_pll|altera_pll_i|general[0].gpll~PLL_OUTPUT_COUNTER|divclk	-85390.465
5	The_System|system_pll|sys_pll|altera_pll_i|general[0].gpll~FRACTIONAL_PLL|vcoph[0]	N/A
6	The_System|system_pll|sys_pll|altera_pl...neral[0].gpll~PLL_OUTPUT_COUNTER|divclk	0.000
7	The_System|vga_pio|altera_pll_i|general[0].gpll~FRACTIONAL_PLL|vcoph[0]	N/A
8	The_System|vga_pio|altera_pll_i|general[0].gpll~PLL_OUTPUT_COUNTER|divclk	-2.110
9	clk_vga	-152.145

Figure 21. Multicorner Timing Analysis Summary

Since the VGA is in a separate clock domain (framerate locked), it is likely not part of the critical path. The Multicorner Timing Analysis Summary shows the worst slack in the m10k_pll domain, which clocks the iterators and M10K blocks. Thus, it is most likely that the critical path occurs on the iteration computation pipeline. In particular, it includes several multiplies, adds, subtracts, and comparisons on mag_squared, z_r, z_i and counts for the done logic for determining when an iteration finishes. All the comparisons and arithmetic is done combinationally within a cycle, so this chain of operations is likely the longest delay.

Conclusion and Future Work

Patience and power is the moral of this lab. Duplicating hardware to parallelize work proved for performance boosts. Future work would include implementing a pipelined solution where we break out iterator work into two stages. Where we have reduced render time by distribution of work can only go so far, so increasing throughput would prove for sub 10 milliseconds rendering times. Additionally, adjusting clock speed, while meeting timing constraints, would be another step forward in unlocking the power of fast rendering. A different parallelization scheme that utilizes more solvers—taking advantage of the unused DSP blocks—would also likely be fruitful.

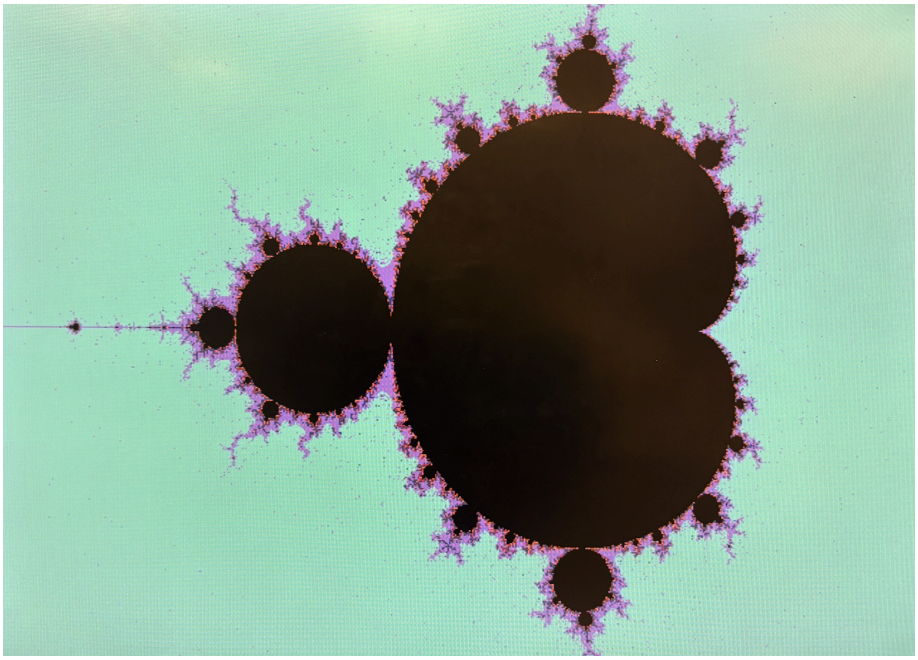


Figure 22: Final VGA visualization of our system.

Work Distribution and AI Citation

Arnav, Nikhil, and Zarif worked together on all parts of the lab. We frequently met outside of our lab sections, and most of our work was done in person in the lab. Nikhil gave extra focus to implementing the state machine to support multiple iterators, while Arnav and Zarif simultaneously focused on implementing the user interface. Outside of the lab, we communicated frequently to ensure that we were all aware of the progress that had been made and what the immediate next steps were. AI was occasionally used to provide ideas for debugging and graphing, but all code, and the contents of this report, were written by us. AI was also occasionally used to provide information on properties of the Mandelbrot set, such as the

main cardioid and the period-2 bulb. The results of these prompts were entirely math, and the corresponding code was written ourselves.

Appendix A: Verilog Implementation (pg 32 - 49)

```
module DE1_SoC_Computer (
        ///////////////////////////////////
        // FPGA Pins
        ///////////////////////////////////

        // Clock pins
        CLOCK_50,
        CLOCK2_50,
        CLOCK3_50,
        CLOCK4_50,

        // ADC
        ADC_CS_N,
        ADC_DIN,
        ADC_DOUT,
        ADC_SCLK,

        // Audio
        AUD_ADCDAT,
        AUD_ADCLRCK,
        AUD_BCLK,
        AUD_DACDAT,
        AUD_DACLRCK,
        AUD_XCK,

        // SDRAM
        DRAM_ADDR,
        DRAM_BA,
        DRAM_CAS_N,
        DRAM_CKE,
        DRAM_CLK,
        DRAM_CS_N,
        DRAM_DQ,
        DRAM_LDQM,
        DRAM_RAS_N,
        DRAM_UDQM,
        DRAM_WE_N,

        // I2C Bus for Configuration of the Audio and Video-In Chips
        FPGA_I2C_SCLK,
        FPGA_I2C_SDAT,

        // 40-Pin Headers
        GPIO_0,
        GPIO_1,

        // Seven Segment Displays
        HEX0,
        HEX1,
        HEX2,
        HEX3,
        HEX4,
        HEX5,

        // IR
```

```
IRDA_RXD,
IRDA_TXD,

// Pushbuttons
KEY,

// LEDs
LEDR,

// PS2 Ports
PS2_CLK,
PS2_DAT,

PS2_CLK2,
PS2_DAT2,

// Slider Switches
SW,

// Video-In
TD_CLK27,
TD_DATA,
TD_HS,
TD_RESET_N,
TD_VS,

// VGA
VGA_B,
VGA_BLANK_N,
VGA_CLK,
VGA_G,
VGA_HS,
VGA_R,
VGA_SYNC_N,
VGA_VS,

///////////////////////////////////
// HPS Pins
///////////////////////////////////

// DDR3 SDRAM
HPS_DDR3_ADDR,
HPS_DDR3_BA,
HPS_DDR3_CAS_N,
HPS_DDR3_CKE,
HPS_DDR3_CK_N,
HPS_DDR3_CK_P,
HPS_DDR3_CS_N,
HPS_DDR3_DM,
HPS_DDR3_DQ,
HPS_DDR3_DQS_N,
HPS_DDR3_DQS_P,
HPS_DDR3_ODT,
HPS_DDR3_RAS_N,
HPS_DDR3_RESET_N,
HPS_DDR3_RZQ,
HPS_DDR3_WE_N,

// Ethernet
HPS_ENET_GTX_CLK,
HPS_ENET_INT_N,
HPS_ENET_MDC,
HPS_ENET_MDIO,
HPS_ENET_RX_CLK,
HPS_ENET_RX_DATA,
```

```
        HPS_ENET_RX_DV,
        HPS_ENET_TX_DATA,
        HPS_ENET_TX_EN,

        // Flash
        HPS_FLASH_DATA,
        HPS_FLASH_DCLK,
        HPS_FLASH_NCSO,

        // Accelerometer
        HPS_GSENSOR_INT,

        // General Purpose I/O
        HPS_GPIO,

        // I2C
        HPS_I2C_CONTROL,
        HPS_I2C1_SCLK,
        HPS_I2C1_SDAT,
        HPS_I2C2_SCLK,
        HPS_I2C2_SDAT,

        // Pushbutton
        HPS_KEY,

        // LED
        HPS_LED,

        // SD Card
        HPS_SD_CLK,
        HPS_SD_CMD,
        HPS_SD_DATA,

        // SPI
        HPS_SPIM_CLK,
        HPS_SPIM_MISO,
        HPS_SPIM_MOSI,
        HPS_SPIM_SS,

        // UART
        HPS_UART_RX,
        HPS_UART_TX,

        // USB
        HPS_CONV_USB_N,
        HPS_USB_CLKOUT,
        HPS_USB_DATA,
        HPS_USB_DIR,
        HPS_USB_NXT,
        HPS_USB_STP
);

//=======================================================
//  PORT declarations
//=======================================================

///////////////////////////////////
// FPGA Pins
///////////////////////////////////

// Clock pins
input                                               CLOCK_50;
input                                               CLOCK2_50;
input                                               CLOCK3_50;
input                                               CLOCK4_50;
```

```
// ADC
inout                                           ADC_CS_N;
output                                    ADC_DIN;
input                                           ADC_DOUT;
output                                    ADC_SCLK;

// Audio
input                                           AUD_ADCDAT;
inout                                           AUD_ADCLRCK;
inout                                           AUD_BCLK;
output                                    AUD_DACDAT;
inout                                           AUD_DACLRCK;
output                                    AUD_XCK;

// SDRAM
output            [12: 0]  DRAM_ADDR;
output            [ 1: 0]  DRAM_BA;
output                                    DRAM_CAS_N;
output                                    DRAM_CKE;
output                                    DRAM_CLK;
output                                    DRAM_CS_N;
inout                       [15: 0]  DRAM_DQ;
output                                    DRAM_LDQM;
output                                    DRAM_RAS_N;
output                                    DRAM_UDQM;
output                                    DRAM_WE_N;

// I2C Bus for Configuration of the Audio and Video-In Chips
output                                    FPGA_I2C_SCLK;
inout                                           FPGA_I2C_SDAT;

// 40-pin headers
inout                       [35: 0]  GPIO_0;
inout                       [35: 0]  GPIO_1;

// Seven Segment Displays
output            [ 6: 0]  HEX0;
output            [ 6: 0]  HEX1;
output            [ 6: 0]  HEX2;
output            [ 6: 0]  HEX3;
output            [ 6: 0]  HEX4;
output            [ 6: 0]  HEX5;

// IR
input                                           IRDA_RXD;
output                                    IRDA_TXD;

// Pushbuttons
input                       [ 3: 0]  KEY;

// LEDs
output            [ 9: 0]  LEDR;

// PS2 Ports
inout                                           PS2_CLK;
inout                                           PS2_DAT;

inout                                           PS2_CLK2;
inout                                           PS2_DAT2;

// Slider Switches
input                       [ 9: 0]  SW;

// Video-In
```

```
input                                           TD_CLK27;
input                   [ 7: 0]  TD_DATA;
input                                           TD_HS;
output                                  TD_RESET_N;
input                                           TD_VS;

// VGA
output          [ 7: 0]  VGA_B;
output                                  VGA_BLANK_N;
output                                  VGA_CLK;
output          [ 7: 0]  VGA_G;
output                                  VGA_HS;
output          [ 7: 0]  VGA_R;
output                                  VGA_SYNC_N;
output                                  VGA_VS;



////////////////////////////////////
// HPS Pins
////////////////////////////////////

// DDR3 SDRAM
output          [14: 0]  HPS_DDR3_ADDR;
output          [ 2: 0]  HPS_DDR3_BA;
output                                  HPS_DDR3_CAS_N;
output                                  HPS_DDR3_CKE;
output                                  HPS_DDR3_CK_N;
output                                  HPS_DDR3_CK_P;
output                                  HPS_DDR3_CS_N;
output          [ 3: 0]  HPS_DDR3_DM;
inout                   [31: 0]  HPS_DDR3_DQ;
inout                   [ 3: 0]  HPS_DDR3_DQS_N;
inout                   [ 3: 0]  HPS_DDR3_DQS_P;
output                                  HPS_DDR3_ODT;
output                                  HPS_DDR3_RAS_N;
output                                  HPS_DDR3_RESET_N;
input                                           HPS_DDR3_RZQ;
output                                  HPS_DDR3_WE_N;

// Ethernet
output                                  HPS_ENET_GTX_CLK;
inout                                           HPS_ENET_INT_N;
output                                  HPS_ENET_MDC;
inout                                           HPS_ENET_MDIO;
input                                           HPS_ENET_RX_CLK;
input                   [ 3: 0]  HPS_ENET_RX_DATA;
input                                           HPS_ENET_RX_DV;
output          [ 3: 0]  HPS_ENET_TX_DATA;
output                                  HPS_ENET_TX_EN;

// Flash
inout                   [ 3: 0]  HPS_FLASH_DATA;
output                                  HPS_FLASH_DCLK;
output                                  HPS_FLASH_NCSO;

// Accelerometer
inout                                           HPS_GSENSOR_INT;

// General Purpose I/O
inout                   [ 1: 0]  HPS_GPIO;

// I2C
inout                                           HPS_I2C_CONTROL;
inout                                           HPS_I2C1_SCLK;
```

```
inout                                               HPS_I2C1_SDAT;
inout                                               HPS_I2C2_SCLK;
inout                                               HPS_I2C2_SDAT;

// Pushbutton
inout                                               HPS_KEY;

// LED
inout                                               HPS_LED;

// SD Card
output                                      HPS_SD_CLK;
inout                                               HPS_SD_CMD;
inout                       [ 3: 0]   HPS_SD_DATA;

// SPI
output                                      HPS_SPIM_CLK;
input                                               HPS_SPIM_MISO;
output                                      HPS_SPIM_MOSI;
inout                                               HPS_SPIM_SS;

// UART
input                                               HPS_UART_RX;
output                                      HPS_UART_TX;

// USB
inout                                               HPS_CONV_USB_N;
input                                               HPS_USB_CLKOUT;
inout                       [ 7: 0]   HPS_USB_DATA;
input                                               HPS_USB_DIR;
input                                               HPS_USB_NXT;
output                                      HPS_USB_STP;

//=======================================================
//  REG/WIRE declarations
//=======================================================

wire [15: 0] hex3_hex0;
assign HEX4 = 7'b1111111;
assign HEX5 = 7'b1111111;
HexDigit Digit0(HEX0, hex3_hex0[3:0]);
HexDigit Digit1(HEX1, hex3_hex0[7:4]);
HexDigit Digit2(HEX2, hex3_hex0[11:8]);
HexDigit Digit3(HEX3, hex3_hex0[15:12]);

// VGA clock and reset lines
wire vga_pll_lock;
wire vga_pll;
wire vga_reset;

// M10k memory control and data
wire [7:0]  M10k_out      [23:0]; // 24 iterators each of 8 bit wide output
reg  [7:0]  write_data    [23:0]; // 24 iterators each of 8 bit wide input
reg  [18:0] write_address [23:0]; // 24 iterators each of 19 bit address
reg  [23:0] write_enable;              // 1 bit enables for each of 24 iterators

// X/y --> real/imag variables
reg         [9:0]  x_coord [23:0]; // 24 coordinates each of 10 bits for each iterator to write x
reg        [9:0]  y_coord [23:0]; // 24 coordinates each of 10 bits for each iterator to write y
reg signed [26:0] c_r     [23:0]; // 24 coordinates each of 10 bits for each iterator to write c_r
reg signed [26:0] c_i     [23:0]; // 24 coordinates each of 10 bits for each iterator to write c_i

// Iterator signals
wire                        [23:0] done;                   // Inidicates when iterations complete
wire signed     [26:0] mag_squared   [23:0];    // Used to determine end of iterations
```

```
wire                        [15:0] count                [23:0];    // How many iteration occured before end conditions?
reg             [23:0] status;                  // all 1s when all iterators done. used for render time

wire [7:0] colors [15:0];
assign colors[0]  = 8'b_111_000_00; // red
assign colors[1]  = 8'b_000_111_00; // green
assign colors[2]  = 8'b_111_111_00; // yellow
assign colors[3]  = 8'b_000_000_11; // blue
assign colors[4]  = 8'b_011_111_00; // green
assign colors[5]  = 8'b_100_000_00; // brown
assign colors[6]  = 8'b_001_110_11; // cyan
assign colors[7]  = 8'b_111_011_00; // orange
assign colors[8]  = 8'b_000_001_11; // dark blue
assign colors[9]  = 8'b_110_000_11; // purple
assign colors[10] = 8'b_001_111_00; // green
assign colors[11] = 8'b_111_100_00; // red
assign colors[12] = 8'b_000_000_00; // black
assign colors[13] = 8'b_111_111_11; // white
assign colors[14] = 8'b_010_101_01; // gray
assign colors[15] = 8'b_101_010_10; // gray

// Reset the VGA screen if FPGA Key1 pressed
assign vga_reset = ~KEY[1];

integer ii; // iteratior i index variable
always @(posedge M10k_pll) begin
         // On a Mandebrot full screen reset
         if (~KEY[0]) begin
        // Iterate through all 24 iterators
                  for (ii = 0; ii < 24; ii = ii + 1) begin
                           write_enable[ii] <= 1'b0;            // Prevent unintentional writing
                           x_coord[ii] <= 10'd0 + (ii % 8);     // Set each iterator to init of respective pattern x
                           y_coord[ii] <= 10'd0 + (ii / 8);       // Set each iterator to init of respective pattern y
                           c_r[ii] <= c_r_ref[26:0] + (ii % 8) * (c_r_step >>> 3); // Set each iterator to init of respective pattern c_r
                           c_i[ii] <= c_i_ref[26:0] - (ii / 8) * (c_i_step / 3);   // Set each iterator to init of respective pattern c_i
                           status[ii] <= 1'b0;                                                  // Reset Rendering status of each iterator
                  end
         end
         // On a VGA reset
         else if (~KEY[1]) begin
                  // Only reset the status of each iterator, which is used to determine when to stop rendering and start the timer
                  for (ii = 0; ii < 24; ii = ii + 1) begin
                           status[ii] <= 1'b0;
                  end
         end
         // Render loop
         else begin
                  // Iterate through all 24 iterators
                  for (ii = 0; ii < 24; ii = ii + 1) begin
                           // Avoid unintentional memory writes
                           write_enable[ii] <= 1'b0;
                           // While iterators are rendering
                           if (done[ii]) begin
                                    write_enable[ii] <= 1'b1;
                  // Enable writing to M10k memory
                                    write_address[ii] <= 19'd80 * (y_coord[ii] / 3) + (x_coord[ii] >> 3);      // Calculate write address based on x/y
coordinates
                                    // Set color based on how many iteration count
                                    // if count >= max iters OR 1000 iterations default
                                    if (count[ii] >= max_iterations) begin
                                      write_data[ii] <= 8'b_000_000_00 ; // black
                                    end
                                    // if count >= 511 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 1)) begin
                                      write_data[ii] <= 8'b_011_001_00 ; // white
```

```
                                    end
                                    // if count >= 255 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 2)) begin
                                      write_data[ii] <= 8'b_011_001_00 ;
                                    end
                                    // if count >= 127 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 3)) begin
                                      write_data[ii] <= 8'b_101_010_01 ;
                                    end
                                    // if count >= 63 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 4)) begin
                                      write_data[ii] <= 8'b_011_001_01 ;
                                    end
                                    // if count >= 31 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 5)) begin
                                      write_data[ii] <= 8'b_001_001_01 ;
                                    end
                                    // if count >= 15 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 6)) begin
                                      write_data[ii] <= 8'b_011_010_10 ;
                                    end
                                    // if count >= 7 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 7)) begin
                                      write_data[ii] <= 8'b_010_100_10 ;
                                    end
                                    // if count >= 3 (if default == 1000 max iters)
                                    else if (count[ii] >= (max_iterations >>> 8)) begin
                                      write_data[ii] <= 8'b_010_100_10 ;
                                    end
                                    // if count anything else, (if default == 1000 max iters)
                                    else begin
                                      write_data[ii] <= 8'b_010_100_10 ;
                                    end

                                    // Update x/y coords for next pixel render or reset if at end of pattern
                                    // Traverse horizontally, if reach end of pattern (or the column) return to beginning of pattern (or the column)
                                    x_coord[ii] <= (x_coord[ii] == (10'd_632 + (ii % 8))) ? (10'd_0 + (ii % 8)) : (x_coord[ii] + 10'd_8);
                                    // Traverse vertically after reaching end of a pattern (or the row). If reach bottom of screen, return to top of
screen or the row
                                    y_coord[ii] <= (x_coord[ii] == (10'd_632 + (ii % 8))) ? ((y_coord[ii] == (10'd_477 + (ii / 8))) ? (10'd_0 + (ii / 8))
: (y_coord[ii] + 10'd_3)) : y_coord[ii];

                                    // C_ref and C_step values are extracted from HPS
                                    // We also maintain our 24 iterator pattern
                                    // Udate c_r/c_i coords for next pixel render or reset if at end of a pattern
                                    c_r[ii] <= (x_coord[ii] == (10'd_632 + (ii % 8))) ? (c_r_ref[26:0] + (ii % 8)*(c_r_step[26:0] >>> 3)) : (c_r[ii] +
c_r_step[26:0]);
                                    // Traverse vertically after reaching end of a pattern (or the row). If reach bottom of screen, return to top of
screen or the row
                                    c_i[ii] <= (x_coord[ii] == (10'd_632 + (ii % 8))) ? ((y_coord[ii] == (10'd_477 + (ii / 8))) ? (c_i_ref[26:0] - (ii /
8)*(c_i_step[26:0] / 3)) : (c_i[ii] - c_i_step[26:0])) : c_i[ii];

                                    // When we reach bottom right pixel, assing status of iterator to 1
                                    status[ii] <= status[ii] || ((x_coord[ii] == (10'd632 + (ii % 8))) && (y_coord[ii] == (10'd477 + (ii / 8))));
                            end
                    end
            end
end

// Generates g number of iterators
generate
            genvar g;
            for (g=0; g < 24; g = g + 1) begin : IteratorGen

                    // One instance of an iterator
```

```
                    iterator foo(
                              .clk(M10k_pll),
                              .reset(~KEY[0]),
                              .c_r(c_r[g]),
                              .c_i(c_i[g]),
                              .mag_squared(mag_squared[g]),
                              .done(done[g]),
                              .count(count[g]),
                              .max_iterations(max_iterations)
                    );

                    // one instance of an M10K Block
                    M10K_1000_8 pixel_data(
                              .q(M10k_out[g]),
                              .d(write_data[g]),
                              .write_address(write_address[g]),
                              .read_address((19'd_80 * (next_y / 3) + (next_x>>3))),
                              .we(write_enable[g]),
                              .clk(M10k_pll)
                    );

          end
endgenerate

// FPGA ref
wire signed [31:0] c_r_ref; // Coordinate to start iterating from in on real axis
wire signed [31:0] c_i_ref; // Coordinate to start iterating from in on imag axis

// ARM zoom
wire signed [31:0] c_r_step; // How much to increment c_r for each pixel determines zoom level
wire signed [31:0] c_i_step; // How much to increment c_i for each pixel determines zoom level

// Mouse PIO inputs from HPS
wire signed [31:0] x_mouse;  // Unused
wire signed [31:0] y_mouse;           // Unused
wire signed [31:0] zoom_mouse;   // Unused

// M10k memory clock
wire M10k_pll;
wire M10k_pll_locked;

// Wires for connecting VGA driver to memory
wire [9:0] next_x;
wire [9:0] next_y;

wire [16:0] max_iterations;                    // Holds value for max iters from switches
//assign max_iterations = 16'd1000; // Default max iters
assign max_iterations = {6'b0, SW}; // Capture binary number created from switches

//reg signed [31:0] x_mouse_sync;  // Unused, Attempted to double flop
//reg signed [31:0] y_mouse_sync;  // Unused, Attempted to double flop
reg [31:0] reset_iterators_pio;    // FPGA signal to reset values and conditions sent to iterators, sent to HPS to reset ref and step

//always@(posedge M10k_pll) begin
//       x_mouse_sync <= x_mouse;
//       y_mouse_sync <= y_mouse;
//end

// Capture a reset
always @(posedge M10k_pll) begin
          if (~KEY[0]) begin
                    reset_iterators_pio <= 32'd1;
          end else begin
                    reset_iterators_pio <= 32'd0;
          end
```

```
end

vga_driver DUT(
         .clock(vga_pll),
         .reset(vga_reset),
         .color_in(M10k_out[(next_x % 8) + 8*(next_y % 3)]), // Editted to index 24 iterators in speific rectangle pattern
         .next_x(next_x),
         .next_y(next_y),
         .hsync(VGA_HS),
         .vsync(VGA_VS),
         .red(VGA_R),
         .green(VGA_G),
         .blue(VGA_B),
         .sync(VGA_SYNC_N),
         .clk(VGA_CLK),
         .blank(VGA_BLANK_N)
);

wire reset_time; //Render time
reg key0_prev;   // Used to debounce key0
reg key1_prev;      // Used to debounce key1
reg  [31:0] start_rendering_pio; // Start render signal sent to HPS
wire [31:0] done_rendering_pio;  // End render signal sent to HPS
reg done_rendering_reg;

// Start Rendering Time Signal on either system or VGA reset
always @(posedge M10k_pll) begin
    key0_prev <= (KEY[0]);
          key1_prev <= (KEY[1]);
    if (~KEY[0]) begin
        start_rendering_pio <= 32'd0;
        key0_prev           <= 1'b0;
    end else if (~KEY[1]) begin
                    start_rendering_pio <= 32'd0;
                    key1_prev           <= 1'b0;
          end
    else if (~key0_prev && (KEY[0]) || ~key1_prev && (KEY[1])) begin
        start_rendering_pio <= 32'd1; // on a debounced click, start capturing render time
    end
end

// End rendering Time Signal
always @(posedge M10k_pll) begin
    if ((~KEY[0]) || (~KEY[1]))
        done_rendering_reg <= 1'b0;
    else if (status == 24'hffffff) // Check if all iterators set their status to 1
        done_rendering_reg <= 1'b1;
end
assign done_rendering_pio = {31'b0, done_rendering_reg}; // send done signal to HPS to stop recording time

//=======================================================
//  Structural coding
//=======================================================
// From Qsys

Computer_System The_System (
         ////////////////////////////////////
         // FPGA Side
         ////////////////////////////////////
         .vga_pio_locked_export                     (vga_pll_lock),           //       vga_pio_locked.export
         .vga_pio_outclk0_clk                               (vga_pll),              //      vga_pio_outclk0.clk
         .m10k_pll_locked_export                    (M10k_pll_locked),          //      m10k_pll_locked.export
         .m10k_pll_outclk0_clk                      (M10k_pll),           //     m10k_pll_outclk0.clk

         // Global signals
```

```
.system_pll_ref_clk_clk                                         (CLOCK_50),
.system_pll_ref_reset_reset                     (1'b0),

////////////////////////////////////
// PIO Ports
////////////////////////////////////
.done_rendering_external_connection_export(done_rendering_pio),
.start_rendering_external_connection_export(start_rendering_pio),
.mouse_zoom_external_connection_export (zoom_mouse),
.zoom_real_external_connection_export (c_r_step),
.zoom_imag_external_connection_export (c_i_step),
.x_external_connection_export (x_mouse),
.y_external_connection_export (y_mouse),
.reset_time_external_connection_export (reset_time),
.reset_iterators_external_connection_export (reset_iterators_pio),
.c_r_ref_external_connection_export (c_r_ref),
.c_i_ref_external_connection_export (c_i_ref),


////////////////////////////////////
// HPS Side
////////////////////////////////////
// DDR3 SDRAM
.memory_mem_a                       (HPS_DDR3_ADDR),
.memory_mem_ba                      (HPS_DDR3_BA),
.memory_mem_ck                      (HPS_DDR3_CK_P),
.memory_mem_ck_n            (HPS_DDR3_CK_N),
.memory_mem_cke             (HPS_DDR3_CKE),
.memory_mem_cs_n            (HPS_DDR3_CS_N),
.memory_mem_ras_n           (HPS_DDR3_RAS_N),
.memory_mem_cas_n           (HPS_DDR3_CAS_N),
.memory_mem_we_n            (HPS_DDR3_WE_N),
.memory_mem_reset_n         (HPS_DDR3_RESET_N),
.memory_mem_dq                      (HPS_DDR3_DQ),
.memory_mem_dqs             (HPS_DDR3_DQS_P),
.memory_mem_dqs_n           (HPS_DDR3_DQS_N),
.memory_mem_odt             (HPS_DDR3_ODT),
.memory_mem_dm                      (HPS_DDR3_DM),
.memory_oct_rzqin           (HPS_DDR3_RZQ),

// Ethernet
.hps_io_hps_io_gpio_inst_GPIO35     (HPS_ENET_INT_N),
.hps_io_hps_io_emac1_inst_TX_CLK    (HPS_ENET_GTX_CLK),
.hps_io_hps_io_emac1_inst_TXD0      (HPS_ENET_TX_DATA[0]),
.hps_io_hps_io_emac1_inst_TXD1      (HPS_ENET_TX_DATA[1]),
.hps_io_hps_io_emac1_inst_TXD2      (HPS_ENET_TX_DATA[2]),
.hps_io_hps_io_emac1_inst_TXD3      (HPS_ENET_TX_DATA[3]),
.hps_io_hps_io_emac1_inst_RXD0      (HPS_ENET_RX_DATA[0]),
.hps_io_hps_io_emac1_inst_MDIO      (HPS_ENET_MDIO),
.hps_io_hps_io_emac1_inst_MDC               (HPS_ENET_MDC),
.hps_io_hps_io_emac1_inst_RX_CTL    (HPS_ENET_RX_DV),
.hps_io_hps_io_emac1_inst_TX_CTL    (HPS_ENET_TX_EN),
.hps_io_hps_io_emac1_inst_RX_CLK    (HPS_ENET_RX_CLK),
.hps_io_hps_io_emac1_inst_RXD1      (HPS_ENET_RX_DATA[1]),
.hps_io_hps_io_emac1_inst_RXD2      (HPS_ENET_RX_DATA[2]),
.hps_io_hps_io_emac1_inst_RXD3      (HPS_ENET_RX_DATA[3]),

// Flash
.hps_io_hps_io_qspi_inst_IO0(HPS_FLASH_DATA[0]),
.hps_io_hps_io_qspi_inst_IO1(HPS_FLASH_DATA[1]),
.hps_io_hps_io_qspi_inst_IO2(HPS_FLASH_DATA[2]),
.hps_io_hps_io_qspi_inst_IO3(HPS_FLASH_DATA[3]),
.hps_io_hps_io_qspi_inst_SS0(HPS_FLASH_NCSO),
.hps_io_hps_io_qspi_inst_CLK(HPS_FLASH_DCLK),
```

```
        // Accelerometer
        .hps_io_hps_io_gpio_inst_GPIO61      (HPS_GSENSOR_INT),

        //.adc_sclk                         (ADC_SCLK),
        //.adc_cs_n                         (ADC_CS_N),
        //.adc_dout                         (ADC_DOUT),
        //.adc_din                          (ADC_DIN),

        // General Purpose I/O
        .hps_io_hps_io_gpio_inst_GPIO40      (HPS_GPIO[0]),
        .hps_io_hps_io_gpio_inst_GPIO41      (HPS_GPIO[1]),

        // I2C
        .hps_io_hps_io_gpio_inst_GPIO48      (HPS_I2C_CONTROL),
        .hps_io_hps_io_i2c0_inst_SDA         (HPS_I2C1_SDAT),
        .hps_io_hps_io_i2c0_inst_SCL         (HPS_I2C1_SCLK),
        .hps_io_hps_io_i2c1_inst_SDA         (HPS_I2C2_SDAT),
        .hps_io_hps_io_i2c1_inst_SCL         (HPS_I2C2_SCLK),

        // Pushbutton
        .hps_io_hps_io_gpio_inst_GPIO54      (HPS_KEY),

        // LED
        .hps_io_hps_io_gpio_inst_GPIO53      (HPS_LED),

        // SD Card
        .hps_io_hps_io_sdio_inst_CMD(HPS_SD_CMD),
        .hps_io_hps_io_sdio_inst_D0 (HPS_SD_DATA[0]),
        .hps_io_hps_io_sdio_inst_D1 (HPS_SD_DATA[1]),
        .hps_io_hps_io_sdio_inst_CLK(HPS_SD_CLK),
        .hps_io_hps_io_sdio_inst_D2 (HPS_SD_DATA[2]),
        .hps_io_hps_io_sdio_inst_D3 (HPS_SD_DATA[3]),

        // SPI
        .hps_io_hps_io_spim1_inst_CLK                (HPS_SPIM_CLK),
        .hps_io_hps_io_spim1_inst_MOSI       (HPS_SPIM_MOSI),
        .hps_io_hps_io_spim1_inst_MISO       (HPS_SPIM_MISO),
        .hps_io_hps_io_spim1_inst_SS0                (HPS_SPIM_SS),

        // UART
        .hps_io_hps_io_uart0_inst_RX(HPS_UART_RX),
        .hps_io_hps_io_uart0_inst_TX(HPS_UART_TX),

        // USB
        .hps_io_hps_io_gpio_inst_GPIO09      (HPS_CONV_USB_N),
        .hps_io_hps_io_usb1_inst_D0          (HPS_USB_DATA[0]),
        .hps_io_hps_io_usb1_inst_D1          (HPS_USB_DATA[1]),
        .hps_io_hps_io_usb1_inst_D2          (HPS_USB_DATA[2]),
        .hps_io_hps_io_usb1_inst_D3          (HPS_USB_DATA[3]),
        .hps_io_hps_io_usb1_inst_D4          (HPS_USB_DATA[4]),
        .hps_io_hps_io_usb1_inst_D5          (HPS_USB_DATA[5]),
        .hps_io_hps_io_usb1_inst_D6          (HPS_USB_DATA[6]),
        .hps_io_hps_io_usb1_inst_D7          (HPS_USB_DATA[7]),
        .hps_io_hps_io_usb1_inst_CLK         (HPS_USB_CLKOUT),
        .hps_io_hps_io_usb1_inst_STP         (HPS_USB_STP),
        .hps_io_hps_io_usb1_inst_DIR         (HPS_USB_DIR),
        .hps_io_hps_io_usb1_inst_NXT         (HPS_USB_NXT)
);
endmodule // end top level

// Declaration of module, include width and signedness of each input/output
module vga_driver (
         input wire clock,
         input wire reset,
         input [7:0] color_in,
```

```
        output [9:0] next_x,
        output [9:0] next_y,
        output wire hsync,
        output wire vsync,
        output [7:0] red,
        output [7:0] green,
        output [7:0] blue,
        output sync,
        output clk,
        output blank
);

        // Horizontal parameters (measured in clock cycles)
        parameter [9:0] H_ACTIVE    =  10'd_639 ;
        parameter [9:0] H_FRONT     =  10'd_15 ;
        parameter [9:0] H_PULSE             =  10'd_95 ;
        parameter [9:0] H_BACK              =  10'd_47 ;

        // Vertical parameters (measured in lines)
        parameter [9:0] V_ACTIVE    =  10'd_479 ;
        parameter [9:0] V_FRONT     =  10'd_9 ;
        parameter [9:0] V_PULSE             =  10'd_1 ;
        parameter [9:0] V_BACK              =  10'd_32 ;

//      // Horizontal parameters (measured in clock cycles)
//      parameter [9:0] H_ACTIVE    =  10'd_9 ;
//      parameter [9:0] H_FRONT     =  10'd_4 ;
//      parameter [9:0] H_PULSE             =  10'd_4 ;
//      parameter [9:0] H_BACK              =  10'd_4 ;
//      parameter [9:0] H_TOTAL     =  10'd_799 ;
//
//      // Vertical parameters (measured in lines)
//      parameter [9:0] V_ACTIVE    =  10'd_1 ;
//      parameter [9:0] V_FRONT     =  10'd_1 ;
//      parameter [9:0] V_PULSE             =  10'd_1 ;
//      parameter [9:0] V_BACK              =  10'd_1 ;

        // Parameters for readability
        parameter         LOW      = 1'b_0 ;
        parameter         HIGH     = 1'b_1 ;

        // States (more readable)
        parameter         [7:0]    H_ACTIVE_STATE             = 8'd_0 ;
        parameter         [7:0]    H_FRONT_STATE              = 8'd_1 ;
        parameter         [7:0]    H_PULSE_STATE              = 8'd_2 ;
        parameter         [7:0]    H_BACK_STATE               = 8'd_3 ;

        parameter         [7:0]    V_ACTIVE_STATE             = 8'd_0 ;
        parameter         [7:0]    V_FRONT_STATE              = 8'd_1 ;
        parameter         [7:0]    V_PULSE_STATE              = 8'd_2 ;
        parameter         [7:0]    V_BACK_STATE               = 8'd_3 ;

        // Clocked registers
        reg               hysnc_reg ;
        reg               vsync_reg ;
        reg      [7:0]    red_reg ;
        reg      [7:0]    green_reg ;
        reg      [7:0]    blue_reg ;
        reg               line_done ;

        // Control registers
        reg      [9:0]    h_counter ;
        reg      [9:0]    v_counter ;

        reg      [7:0]    h_state ;
```

```
reg      [7:0]    v_state ;

// State machine
always@(posedge clock) begin
         // At reset . . .
         if (reset) begin
                  // Zero the counters
                  h_counter          <= 10'd_0 ;
                  v_counter          <= 10'd_0 ;
                  // States to ACTIVE
                  h_state   <= H_ACTIVE_STATE  ;
                  v_state   <= V_ACTIVE_STATE  ;
                  // Deassert line done
                  line_done          <= LOW ;
         end
         else begin
                  ////////////////////////////////////////////////////////////////////////
                  ////////////////////////// HORIZONTAL //////////////////////////////////
                  ////////////////////////////////////////////////////////////////////////
                  if (h_state == H_ACTIVE_STATE) begin
                           // Iterate horizontal counter, zero at end of ACTIVE mode
                           h_counter <= (h_counter==H_ACTIVE)?10'd_0:(h_counter + 10'd_1) ;
                           // Set hsync
                           hysnc_reg <= HIGH ;
                           // Deassert line done
                           line_done <= LOW ;
                           // State transition
                           h_state <= (h_counter == H_ACTIVE)?H_FRONT_STATE:H_ACTIVE_STATE ;
                  end
                  // Assert done flag, wait here for reset
                  if (h_state == H_FRONT_STATE) begin
                           // Iterate horizontal counter, zero at end of H_FRONT mode
                           h_counter <= (h_counter==H_FRONT)?10'd_0:(h_counter + 10'd_1) ;
                           // Set hsync
                           hysnc_reg <= HIGH ;
                           // State transition
                           h_state <= (h_counter == H_FRONT)?H_PULSE_STATE:H_FRONT_STATE ;
                  end
                  if (h_state == H_PULSE_STATE) begin
                           // Iterate horizontal counter, zero at end of H_FRONT mode
                           h_counter <= (h_counter==H_PULSE)?10'd_0:(h_counter + 10'd_1) ;
                           // Set hsync
                           hysnc_reg <= LOW ;
                           // State transition
                           h_state <= (h_counter == H_PULSE)?H_BACK_STATE:H_PULSE_STATE ;
                  end
                  if (h_state == H_BACK_STATE) begin
                           // Iterate horizontal counter, zero at end of H_FRONT mode
                           h_counter <= (h_counter==H_BACK)?10'd_0:(h_counter + 10'd_1) ;
                           // Set hsync
                           hysnc_reg <= HIGH ;
                           // State transition
                           h_state <= (h_counter == H_BACK)?H_ACTIVE_STATE:H_BACK_STATE ;
                           // Signal line complete at state transition (offset by 1 for synchronous state transition)
                           line_done <= (h_counter == (H_BACK-1))?HIGH:LOW ;
                  end
                  ////////////////////////////////////////////////////////////////////////
                  ////////////////////////// VERTICAL ////////////////////////////////////
                  ////////////////////////////////////////////////////////////////////////
                  if (v_state == V_ACTIVE_STATE) begin
                           // increment vertical counter at end of line, zero on state transition
                           v_counter <= (line_done==HIGH)?((v_counter==V_ACTIVE)?10'd_0:(v_counter + 10'd_1)):v_counter ;
                           // set vsync in active mode
                           vsync_reg <= HIGH ;
                           // state transition - only on end of lines
```

```
                                v_state <= (line_done==HIGH)?((v_counter==V_ACTIVE)?V_FRONT_STATE:V_ACTIVE_STATE):V_ACTIVE_STATE ;
                        end
                        if (v_state == V_FRONT_STATE) begin
                                // increment vertical counter at end of line, zero on state transition
                                v_counter <= (line_done==HIGH)?((v_counter==V_FRONT)?10'd_0:(v_counter + 10'd_1)):v_counter ;
                                // set vsync in front porch
                                vsync_reg <= HIGH ;
                                // state transition
                                v_state <= (line_done==HIGH)?((v_counter==V_FRONT)?V_PULSE_STATE:V_FRONT_STATE):V_FRONT_STATE ;
                        end
                        if (v_state == V_PULSE_STATE) begin
                                // increment vertical counter at end of line, zero on state transition
                                v_counter <= (line_done==HIGH)?((v_counter==V_PULSE)?10'd_0:(v_counter + 10'd_1)):v_counter ;
                                // clear vsync in pulse
                                vsync_reg <= LOW ;
                                // state transition
                                v_state <= (line_done==HIGH)?((v_counter==V_PULSE)?V_BACK_STATE:V_PULSE_STATE):V_PULSE_STATE ;
                        end
                        if (v_state == V_BACK_STATE) begin
                                // increment vertical counter at end of line, zero on state transition
                                v_counter <= (line_done==HIGH)?((v_counter==V_BACK)?10'd_0:(v_counter + 10'd_1)):v_counter ;
                                // set vsync in back porch
                                vsync_reg <= HIGH ;
                                // state transition
                                v_state <= (line_done==HIGH)?((v_counter==V_BACK)?V_ACTIVE_STATE:V_BACK_STATE):V_BACK_STATE ;
                        end

                        ////////////////////////////////////////////////////////////////////////
                        ////////////////////////////////// COLOR OUT ///////////////////////////////
                        ////////////////////////////////////////////////////////////////////////
                        red_reg            <= (h_state==H_ACTIVE_STATE)?((v_state==V_ACTIVE_STATE)?{color_in[7:5],5'd_0}:8'd_0):8'd_0 ;
                        green_reg          <= (h_state==H_ACTIVE_STATE)?((v_state==V_ACTIVE_STATE)?{color_in[4:2],5'd_0}:8'd_0):8'd_0 ;
                        blue_reg  <= (h_state==H_ACTIVE_STATE)?((v_state==V_ACTIVE_STATE)?{color_in[1:0],6'd_0}:8'd_0):8'd_0 ;

                end
        end
        // Assign output values
        assign hsync = hysnc_reg ;
        assign vsync = vsync_reg ;
        assign red = red_reg ;
        assign green = green_reg ;
        assign blue = blue_reg ;
        assign clk = clock ;
        assign sync = 1'b_0 ;
        assign blank = hysnc_reg & vsync_reg ;
        // The x/y coordinates that should be available on the NEXT cycle
        assign next_x = (h_state==H_ACTIVE_STATE)?h_counter:10'd_0 ;
        assign next_y = (v_state==V_ACTIVE_STATE)?v_counter:10'd_0 ;

endmodule


//=============================================================
// M10K module for testing
//=============================================================

module M10K_1000_8(
        output reg [7:0] q,
        input [7:0] d,
        input [18:0] write_address, read_address,
        input we, clk
);

        // 307200 bits divided by 24 iterators
        reg [7:0] mem [12800:0] /* synthesis ramstyle = "no_rw_check, M10K" */;
```

```
        always @ (posedge clk) begin
                if (we) begin
                        mem[write_address] <= d;
                end
                        q <= mem[read_address];
                end
endmodule


//============================================================
// Mandelbrot Iterator
//============================================================

module iterator (clk, reset, c_r, c_i, mag_squared, done, count, max_iterations);
        input              clk;
        input              reset;
        input  signed [26:0] c_r;
        input  signed [26:0] c_i;
        output signed [26:0] mag_squared;
        output reg           done;
        output reg  [15:0]   count;
        input [15:0]         max_iterations;

        reg  signed [26:0] z_r;
        reg  signed [26:0] z_i;
        reg  signed [26:0] z_r_squared;
        reg  signed [26:0] z_i_squared;

        wire signed [26:0] z_r_next;
        wire signed [26:0] z_i_next;
        wire signed [26:0] z_r_squared_next;
        wire signed [26:0] z_i_squared_next;
        wire signed [26:0] temp;

        // z_r_next = z_r^2 - z_i^2 + c_r
        assign z_r_next = z_r_squared - z_i_squared + c_r;

        // z_i_next = 2*z_r*z_i + c_i
        signed_mult mult1(
                .out(temp),
                .a(z_r),
                .b(z_i)
        );
        assign z_i_next = (temp << 1) + c_i;

        // Precompute squares for next cycle
        signed_mult mult2(
                .out(z_i_squared_next),
                .a(z_i_next),
                .b(z_i_next)
        );
        signed_mult mult3(
                .out(z_r_squared_next),
                .a(z_r_next),
                .b(z_r_next)
        );

        // |z|^2 = z_r^2 + z_i^2
        assign mag_squared = z_r_squared_next + z_i_squared_next;

        wire [26:0] cr_plus_1_squared;
        wire [26:0] ci_squared;

        wire [26:0] cr_minus_quarter_squared;
```

```
//        signed_mult cr1_mult(
//                  .out(cr_plus_1_squared),
//                  .a(c_r + 27'sh0800000),
//                  .b(c_r + 27'sh0800000)
//        );
//
//        signed_mult ci_mult(
//                  .out(ci_squared),
//                  .a(c_i),
//                  .b(c_i)
//        );

        /*signed_mult cr_quarter_mult(
                  .out(cr_minus_quarter_squared),
                  .a(c_r - 27'sh0200000),
                  .b(c_r - 27'sh0200000)
        );

        wire [26:0] q;
        assign q = cr_minus_quarter_squared + ci_squared;

        wire [26:0] prod;
        signed_mult cond_mult(
                  .out(prod),
                  .a(q),
                  .b(q + (c_r - 27'sh0200000))
        );*/

        // Check if magnitude of computing sequence exceeds 4, signal done
        always @(*) begin
                  if ((mag_squared > 27'sh2000000) || (z_i > 27'sh1000000) || (z_i < 27'sh7000000) || (z_r > 27'sh1000000) || (z_r < 27'sh7000000))
                            done = 1'b1;
                  // Otherwise if not yet met magnitude of 4, stop at maximum number of iterations
                  else if (count >= max_iterations)
                            done = 1'b1;
                  else
                            done = 1'b0; // else keep going
        end

        always @(posedge clk) begin
                  if (reset || done) begin
                            z_r         <= 27'b0;
                            z_i         <= 27'b0;
                            z_r_squared <= 27'b0;
                            z_i_squared <= 27'b0;
                            count       <= 16'b0;
                  end
                  else begin
                            z_r         <= z_r_next;
                            z_i         <= z_i_next;
                            z_r_squared <= z_r_squared_next;
                            z_i_squared <= z_i_squared_next;

                            // Rectangles to automatically converge to max iterations
                            // Cardioid Largest Rectangle auto done
                            if (c_r <= 27'sh0200000 && c_r >= 27'sh7c00000 && c_i <= 27'sh0400000 && c_i >= 27'sh7c00000) begin
                                      count <= max_iterations;
                            end
                            // Period 2 Circle Smallest Rectangle auto done
                            else if (c_r <= 27'sh00796A0A1 && c_r >= 27'sh007695F60 && c_i <= 27'sh00016A0A0 && c_i >= 27'sh007E95F60) begin
                                      count <= max_iterations;
                            end
/*                          else if (cr_plus_1_squared + ci_squared <= 27'sh0080000) begin
```

```
                                        count <= max_iterations;
                                end
/*                              else if (prod <= (ci_squared >> 2)) begin
                                        count <= 16'd1000;
                                end*/
                                // Otherwise keep counting
                                else begin
                                        count       <= count + 16'd1;
                                end
                        end
                end

endmodule


//=====================================================
// 4.23 Fixed Point Signed Multiplier
//=====================================================

module signed_mult (out, a, b);
output signed [26:0] out;
input  signed [26:0] a;
input  signed [26:0] b;

wire signed [53:0] mult_out;
assign mult_out = a * b;

// Editted for:
// 4.23 fixed point: keep sign bit and bits [48:23]
assign out = {mult_out[53], mult_out[48:23]};
endmodule
```

Appendix B: C Code (pg. 50-56)

```c
////////////////////////////////////////////////////////////////////////
// Mandelbrot Mouse Controller + Render Timer (threaded)
// Compile: gcc mandel.c -o mandel -lm -lpthread
////////////////////////////////////////////////////////////////////////
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/mman.h>
#include <sys/time.h>
#include <math.h>
#include <pthread.h> // Import pthread for threading

// ========================================================
// ===== Begin 4.23 Fix <--> Float Conversion Macro =====
// ========================================================
typedef int fix;
#define float2fix(a)     ((fix)((a) * (1 << 23)))
#define fix2float(a)     (((float)(a)) / (float)(1 << 23))
// ========================================================
// ======= End 4.23 Fix <--> Float Conversion Macro =====
// ========================================================

// ========================================================
// ======= Begin Open Mouse Device Reading =====
// ========================================================
const char *pDevice = "/dev/input/mice";
// ========================================================
// ======= End Open Mouse Device Reading =====
// ========================================================

#define FPGA_AXI_BASE           0xC0000000
#define FPGA_AXI_SPAN           0x00001000

// ========================================================
// ===== Begin PIO Port Address Offsets =====
// ========================================================
#define START_RENDERING_PIO     0x00
#define DONE_RENDERING_PIO      0x10
#define MOUSE_ZOOM_PIO_WRITE    0x20
#define MOUSE_X_PIO_WRITE       0x30
#define MOUSE_Y_PIO_WRITE       0x40
#define ZOOM_REAL_PIO_WRITE     0x50
#define ZOOM_IMAG_PIO_WRITE     0x60
#define RESET_TIME_PIO_WRITE    0x70
#define RESET_ITERATORS_PIO     0x80
#define C_R_REF_PIO             0x90
#define C_I_REF_PIO             0xA0
```

```
// ========================================================
// ===== End PIO Port Address Offsets =======
// ========================================================

// ========================================================
// ===== Begin Constant Init Values ======
// ========================================================
#define ZOOM_REAL_INIT   (24.0f / 640.0f)
#define ZOOM_IMAG_INIT   (6.0f / 480.0f)
#define C_R_REF_INIT     (-2.0f)
#define C_I_REF_INIT     (1.0f)
float zoom_real = ZOOM_REAL_INIT;
float zoom_imag = ZOOM_IMAG_INIT;
float c_r_ref   = C_R_REF_INIT;
float c_i_ref   = C_I_REF_INIT;
#define PAN_SCALE        (1.0f / 8.0f)
#define PAN_MAX        8
#define PAN_MIN        -8
// ========================================================
// ======= End Constant Init Values ========
// ========================================================

void *h2p_virtual_base;

// ========================================================
// ====== Begin PIO Address pointer Declarations =====
// ========================================================
volatile unsigned int *mouse_zoom_write_ptr     = NULL;
volatile unsigned int *mouse_x_write_ptr        = NULL;
volatile unsigned int *mouse_y_write_ptr        = NULL;
volatile unsigned int *zoom_real_write_ptr      = NULL;
volatile unsigned int *zoom_imag_write_ptr      = NULL;
volatile unsigned int *start_rendering_pio      = NULL;
volatile unsigned int *done_rendering_pio       = NULL;
volatile unsigned int *reset_time_pio_write_ptr = NULL;
volatile unsigned int *reset_iterators_pio_ptr  = NULL;
volatile unsigned int *c_r_ref_write_ptr        = NULL;
volatile unsigned int *c_i_ref_write_ptr        = NULL;
// ========================================================
// ===== End PIO Address pointer Declarations =====
// ========================================================

// ========================================================
// ===== Begin Mouse Input Thread =====
// ========================================================
void *mouse_thread(void *arg)
{
    int mouse_fd;
    // if ((mouse_fd = open(MOUSE_DEVICE, O_RDONLY | O_NONBLOCK)) == -1) {
    //     printf("ERROR: could not open %s\n", MOUSE_DEVICE);
    //     return NULL;
    // }

    // Open Mouse file for reading, with error exception
    // Used O_NONBLOCK to avoid blocking other threads on a read
    mouse_fd = open(pDevice, O_RDWR | O_NONBLOCK);
```

```
    if(mouse_fd == -1)
    {
        printf("ERROR Opening %s\n", pDevice);
        return NULL;
    }
    // The Data read form mouse
    unsigned char data[3];
    int prev_left  = 0;
    int prev_right = 0;
    while (1)
    {
        // ------------------------------------------------------------------------------------------
        // ------ begin FPGA KEY[0] reset ----------
        // ------------------------------------------------------------------------------------------
        if (*reset_iterators_pio_ptr == 1) {

            zoom_real = ZOOM_REAL_INIT;            // Reset real axis step size to init
            zoom_imag = ZOOM_IMAG_INIT;            // Reset imag axis step size to init
            c_r_ref = C_R_REF_INIT;                // Reset real axis starting rendiner point to init
            c_i_ref = C_I_REF_INIT;                // Reset imag axis starting rendiner point to init
            *zoom_real_write_ptr = (unsigned int)float2fix(zoom_real); // Send init real step size val to FPGA
            *zoom_imag_write_ptr = (unsigned int)float2fix(zoom_imag); // Send init imag step size val to FPGA
            *c_r_ref_write_ptr = (unsigned int)float2fix(c_r_ref); // Send init real starting rendering point size val to FPGA
            *c_i_ref_write_ptr = (unsigned int)float2fix(c_i_ref); // Send init imag stepsize val to FPGA
            printf("Top left: (%.6f, %.6f)\n", c_r_ref, c_i_ref);   // Read top left pixel location of screen (complex plane)
            printf("Top right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref); // Read top right pixel location of screen (complex
plane)
            printf("Bottom left: (%.6f, %.6f)\n", c_r_ref, c_i_ref - 480*zoom_imag/4); // Read bottom left pixel location of screen
(complex plane)
            printf("Bottom right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref - 480*zoom_imag/4); // Read bottom right pixel
location of screen (complex plane)

            // After reset wait for signal to go low after a little bit before allowing other actions
            while (*reset_iterators_pio_ptr == 1){
                usleep(1000);
            }
        }
        // ------------------------------------------------------------
        // ------ end begin FPGA KEY[0] or KEY[1] reset -----------
        // ------------------------------------------------------------

        // ------------------------------------------------------------
        // --------- begin processing mouse inputs ---------
        // ------------------------------------------------------------
        else if (read(mouse_fd, data, sizeof(data)) > 0)
        {
            // Mask left and right Button clicks
            int left  = data[0] & 0x1;
            int right = data[0] & 0x2;

            //Track positive or negative movement from mouse
            signed char dx_raw = (signed char)data[1];
            signed char dy_raw = (signed char)data[2];

            // Clamp to avoid huge jumps on fast swipes
            int dx = (int)dx_raw;
```

```
            int dy = (int)dy_raw;

            // If greater or less than max or min pans, clamp to max or min
            if (dx > PAN_MAX) dx = PAN_MAX;
            if (dx < PAN_MIN) dx = PAN_MIN;
            if (dy > PAN_MAX) dy = PAN_MAX;
            if (dy < PAN_MIN) dy = PAN_MIN;
            // ----------------------------------------------------------
            // -------- begin Panning ---------
            // ----------------------------------------------------------
            // If any movement, pan screen
            if (dx != 0 || dy != 0) {
                // Scale the moement then incremenet the reference point
                c_r_ref += (float)dx * PAN_SCALE * zoom_real;
                c_i_ref += (float)dy * PAN_SCALE * zoom_imag;
                // Write back to memory
                *c_r_ref_write_ptr = (unsigned int)float2fix(c_r_ref);
                *c_i_ref_write_ptr = (unsigned int)float2fix(c_i_ref);
                printf("Top left: (%.6f, %.6f)\n", c_r_ref, c_i_ref);                      // Read top left pixel location of screen
(complex plane)
                printf("Top right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/8, c_i_ref);   // Read top right pixel location of screen
(complex plane)
                printf("Bottom left: (%.6f, %.6f)\n", c_r_ref, c_i_ref - 480*zoom_imag/3); // Read bottom left pixel location of screen
(complex plane)
                printf("Bottom right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/8, c_i_ref - 480*zoom_imag/3); // Read bottom right pixel
location of screen (complex plane)
            }
            // ----------------------------------------------------------
            // ------ End Panning ----------
            // ----------------------------------------------------------

            // ----------------------------------------------------------
            // ------ Being Zoom In ----------
            // ----------------------------------------------------------

            // On left click zoom in
            if (left && !prev_left) { // Only trigger on initial button press, debounced
                *reset_time_pio_write_ptr = 1; // Signal to FPGA to reset timer while we update values in memory
                zoom_real /= 2.0f; // Zoom real axis in by factor of 2
                zoom_imag /= 2.0f; // Zoom imag axis in by factor of 2

                // Write back to memory, send new ref and steps to FPGA
                *zoom_real_write_ptr = (unsigned int)float2fix(zoom_real);
                *zoom_imag_write_ptr = (unsigned int)float2fix(zoom_imag);
                *c_r_ref_write_ptr = (unsigned int)float2fix(c_r_ref);
                *c_i_ref_write_ptr = (unsigned int)float2fix(c_i_ref);
                printf("Top left: (%.6f, %.6f)\n", c_r_ref, c_i_ref);                      // Read top left pixel location of screen
(complex plane) on pan
                printf("Top right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref);    // Read top right pixel location of screen
(complex plane) on pan
                printf("Bottom left: (%.6f, %.6f)\n", c_r_ref, c_i_ref - 480*zoom_imag/4);  // Read bottom left pixel location of screen
(complex plane) on pan
                printf("Bottom right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref - 480*zoom_imag/4); // Read bottom right pixel
location of screen (complex plane) on pan
                *reset_time_pio_write_ptr = 0; // Release FPGA timer reset signal after updates to memory
            }
```

```
            // ---------------------------------------------------------
            // -------- End Zoom In ---------
            // ---------------------------------------------------------

            // ---------------------------------------------------------
            // ------ Being Zoom Out ---------
            // ---------------------------------------------------------

            // On right click zoom out
            else if (right && !prev_right) { // Only trigger on initial button press, debounced
                float new_real = zoom_real * 2.0f; // Zoom real axis out by factor of 2
                float new_imag = zoom_imag * 2.0f; // Zoom imag axis out by factor of 2
                *reset_time_pio_write_ptr = 1; // Signal to FPGA to reset timer while we update values in memory

                // Clamp Zoom out
                if (new_real > ZOOM_REAL_INIT || new_imag > ZOOM_IMAG_INIT) {
                    zoom_real = ZOOM_REAL_INIT;
                    zoom_imag = ZOOM_IMAG_INIT;
                } else {
                    zoom_real = new_real;
                    zoom_imag = new_imag;
                }

                // Write back to memory, send new ref and steps to FPGA
                *zoom_real_write_ptr = (unsigned int)float2fix(zoom_real);
                *zoom_imag_write_ptr = (unsigned int)float2fix(zoom_imag);
                *c_r_ref_write_ptr   = (unsigned int)float2fix(c_r_ref);
                *c_i_ref_write_ptr   = (unsigned int)float2fix(c_i_ref);
                printf("Top left: (%.6f, %.6f)\n", c_r_ref, c_i_ref);                                  // Read top left pixel location of
screen (complex plane) on pan
                printf("Top right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref);         // Read top right pixel location of
screen (complex plane) on pan
                printf("Bottom left: (%.6f, %.6f)\n", c_r_ref, c_i_ref - 480*zoom_imag/4);       // Read bottom left pixel location of
screen (complex plane) on pan
                printf("Bottom right: (%.6f, %.6f)\n", c_r_ref + 640*zoom_real/4, c_i_ref - 480*zoom_imag/4); // Read top left pixel
location of screen (complex plane) on pan
                *reset_time_pio_write_ptr = 0; // Release FPGA timer reset signal after updates to memory
            }
            // ---------------------------------------------------------
            // -------- End Zoom Out ---------
            // ---------------------------------------------------------

            // Track previous button press to avoid multiple zooms on long presses
            prev_left  = left;
            prev_right = right;
        }
        else {
            usleep(1000);
        }
    }
    close(mouse_fd); // Good practice, never reaches
    return NULL;
}
// ============================================================================
// ======= End Mouse Input Thread =====
// ============================================================================
```

```c
// ================================================================================
// ======= Begin Timer Thread =====
// ================================================================================
void *timer_thread(void *arg)
{
    struct timeval t1, t2;
    double elapsedTime;

    while (1)
    {
        printf("Waiting for start rendering signal...\n");
        while (*start_rendering_pio == 0) { continue; }
        printf("Start rendering received.\n");

        gettimeofday(&t1, NULL);

        printf("Waiting for done rendering signal...\n");
        while (*done_rendering_pio == 0) { continue; }

        gettimeofday(&t2, NULL);
        elapsedTime  = (t2.tv_sec  - t1.tv_sec)  * 1000.0;
        elapsedTime += (t2.tv_usec - t1.tv_usec) / 1000.0;
        printf("Render time: %.6f ms\n", elapsedTime);

        // Wait for start to go low before looping
        while (*start_rendering_pio == 1) { continue; }
    }

    return NULL;
}
// ================================================================
// ===== End Timer Thread =======
// ================================================================================

int main(void)
{
    int mem_fd;
    if ((mem_fd = open("/dev/mem", (O_RDWR | O_SYNC))) == -1) {
        printf("ERROR: could not open /dev/mem\n");
        return 1;
    }

    h2p_virtual_base = mmap(NULL, FPGA_AXI_SPAN,
                            (PROT_READ | PROT_WRITE),
                            MAP_SHARED, mem_fd, FPGA_AXI_BASE);
    if (h2p_virtual_base == MAP_FAILED) {
        printf("ERROR: mmap() failed\n");
        close(mem_fd);
        return 1;
    }

    // ================================================================================
    // ===== Begin PIO Pointer Addressing =======
    // ================================================================================
    mouse_zoom_write_ptr     = (unsigned int *)(h2p_virtual_base + MOUSE_ZOOM_PIO_WRITE);
```

```
    mouse_x_write_ptr        = (unsigned int *)(h2p_virtual_base + MOUSE_X_PIO_WRITE);
    mouse_y_write_ptr        = (unsigned int *)(h2p_virtual_base + MOUSE_Y_PIO_WRITE);
    zoom_real_write_ptr      = (unsigned int *)(h2p_virtual_base + ZOOM_REAL_PIO_WRITE);
    zoom_imag_write_ptr      = (unsigned int *)(h2p_virtual_base + ZOOM_IMAG_PIO_WRITE);
    start_rendering_pio      = (unsigned int *)(h2p_virtual_base + START_RENDERING_PIO);
    done_rendering_pio       = (unsigned int *)(h2p_virtual_base + DONE_RENDERING_PIO);
    reset_time_pio_write_ptr = (unsigned int *)(h2p_virtual_base + RESET_TIME_PIO_WRITE);
    reset_iterators_pio_ptr  = (unsigned int *)(h2p_virtual_base + RESET_ITERATORS_PIO);
    c_r_ref_write_ptr        = (unsigned int *)(h2p_virtual_base + C_R_REF_PIO);
    c_i_ref_write_ptr        = (unsigned int *)(h2p_virtual_base + C_I_REF_PIO);
    // ================================================================================
    // ===== End PIO Pointer Addressing ======
    // ================================================================================

    // ========================================================
    // ===== Initialize Mem vars =====
    // ========================================================
    *zoom_real_write_ptr = (unsigned int)float2fix(zoom_real);
    *zoom_imag_write_ptr = (unsigned int)float2fix(zoom_imag);
    *c_r_ref_write_ptr   = (unsigned int)float2fix(c_r_ref);
    *c_i_ref_write_ptr   = (unsigned int)float2fix(c_i_ref);
    // ========================================================
    // ===== Initialize Mem vars ======
    // ========================================================

    // Launch threads
    pthread_t t_mouse, t_timer;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);

    // Startup threads
    pthread_create(&t_mouse, &attr, mouse_thread, NULL);
    pthread_create(&t_timer, &attr, timer_thread, NULL);

    // unreachable good practise
    pthread_join(t_mouse, NULL);
    pthread_join(t_timer, NULL);
    munmap(h2p_virtual_base, FPGA_AXI_SPAN);
    close(mem_fd);
    return 0;
}
```